Recoverability Analysis of Language-Model-Generated Shell Commands, Deployed in the SynapseOS Conversational Interface

by

Alexandra Sulit

Willard Soriano

Steven Evian Lozano

A Thesis Proposal Submitted to the Department of Computer Science
in Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science

Mapúa University – Makati
September 2026

APPROVAL PAGE

[To be inserted upon approval.]

ACKNOWLEDGEMENT

[To be inserted prior to final submission.]

TABLE OF CONTENTS

TITLE PAGE i

APPROVAL PAGE ii

ACKNOWLEDGEMENT iii

TABLE OF CONTENTS iv

LIST OF TABLES vi

LIST OF FIGURES vii

Chapter 1: INTRODUCTION 1

1.1 Background of the Study 1

1.2 Gap and Opportunity 2

1.3 Statement of the Problem 4

1.4 Objectives of the Study 5

1.5 Contributions of the Study 6

1.6 Significance of the Study 10

1.7 Scope and Limitations of the Study 11

Chapter 2: REVIEW OF RELATED LITERATURE 15

2.1 Introduction to the Review 15

2.2 Theoretical Framework 17

2.3 Historical Foundations of Natural-Language Interfaces 21

2.4 Natural-Language-to-Shell Translation 24

2.5 Command Safety, Static Analysis, and Recovery 25

2.6 Graphical and Computer-Using Agents 32

2.7 Accessibility and Speech-Driven Interaction 35

2.8 Evaluation Environments and Benchmarks 36

2.9 Synthesis: Comparative Analysis and Research Gap 37

2.10 Review of the Methodological Literature 44

2.11 Summary 46

Chapter 3: METHODOLOGY 49

Conceptual Framework 50

System Environment Specifications 52

Section 1: Dataset 55

Section 2: Procedure / Experiment 68

Section 3: System Testing 86

REFERENCES **109**

APPENDICES **116**

LIST OF TABLES

Table 1.1: Components adopted rather than built, and why 6

Table 2.1: Approaches to the safety of shell commands, compared on the properties the study’s algorithm combines 31

Table 2.2: Comparative analysis of surveyed systems on conversational and LLM-mediated interfaces for personal computing 38

Table 3.1: Study evaluation machines 53

Table 3.2: SynapseOS minimum deployment requirements 53

Table 3.3: Metrics for the recoverability algorithm, and what each is for 88

Table 3.4: Paired comparisons of the effect analysis against each list on the held-out corpus, with Holm-corrected significance 92

Table 3.5: Composition and resolution ablation — the full analysis against the same rule table with composition, resolution, or both disabled 98

Table C.1: Custom cross-platform task suite — core tasks 119

Table C.2: Boundary tasks — outside shell expressibility 121

LIST OF FIGURES

Figure 2.1: The four-layer interaction pipeline for language-mediated computing interfaces 19

Figure 3.1: Conceptual framework: the recoverability analysis and the platform that deploys it 52

Chapter 1

INTRODUCTION

1.1 Background of the Study

Ask a person to gather every photo they took last June into a single folder and they will do it — opening a file browser, sorting by date, selecting, dragging. Ask a computer the same thing, in those same words, and for most of the history of personal computing it could not. The instruction has had to be translated first into the machine's own vocabulary: a path, a menu, a sequence of clicks, or a command typed with exact syntax.

The wish to remove that translation, and instruct a computer in ordinary language, is old: the Berkeley UNIX Consultant, an early system for conversing with an operating system, was operational by the late 1980s [49]. Large language models (LLMs) and multimodal foundation models[1] have revived it with far greater capability. They translate natural language into Bash commands with usable accuracy [13, 47], and computer-using agents[2] built on them operate desktop, mobile, and web applications [53, 54, 56, 57]. Increasingly, these systems act by running commands on a real machine, and so on a person's own files.

In writing the command for a person who could not, language models also move the risk. The person no longer has to know the command, but neither can they check it, and when a command they did not write and cannot read is wrong, it is their files that are lost. Whether that loss can be undone depends on what was saved before the command ran.

Knowing what to save is not simple, because shell commands are programs. Their targets may be hidden inside other commands, as in `sudo rm`, `find -exec`, or `xargs`, or computed only while they run, as in `rm $(ls *.log)`, so what a command will change cannot be read from its name. Determining that before the command runs, and saving beforehand exactly what it would destroy, is therefore a problem of program analysis rather than of interface design. This study addresses that problem, using a conversational interface layer for the Linux desktop as the setting in which the analysis is deployed and validated.

1.2 Gap and Opportunity

Tools that run commands written by a language model already try to protect the people who use them, but none of them works out, before a command runs, what it would destroy and saves that so the command can be undone. The systems surveyed in Chapter 2 that run generated commands protect in one of two ways. Most guard by *permission*: they match the command against a list of patterns judged dangerous, then allow it, refuse it, or put it to the user, as in the dangerous-command check of OpenAI's Codex CLI [32] and in cc-safety-net, a guard used with several AI coding agents [21]. A few shipping systems add coarse recovery: Alibaba Cloud's ANOLISA, a Linux distribution for AI agents, snapshots an agent's workspace before key operations or after each round of conversation and can roll back to any snapshot [1], but a rollback restores everything since the snapshot, not what one command destroyed.

A list judges a command by its name and arguments, so it misses what the command touches through another command or through a target computed at run time, and neither a list nor a snapshot can say, before a command runs, whether it will lose anything. Designs closer to this study's aim do not close the gap either. NaSh, a design for a natural-language shell, proposes summaries of what a command would affect and selective undo but leaves their mechanism open [13], and *try* contains a command's effects only by running it, for the user to judge [22]. No system in the surveyed

literature, and no shipping tool examined, derives from a generated command's structure, before it runs, what it would destroy and captures exactly that so that the command can be undone; that is the gap this study addresses.

The opportunity is algorithmic. A command's effect can often be derived from the command itself, before it runs, and what it would destroy saved first; where it cannot be derived, the analysis can say so and ask. This study builds that analysis, on a few terms used throughout. An *effect* is one thing a command does to one path — reading, creating, writing, or removing it, or changing its permissions — or to the system's installed packages and running services. A *pre-image* is the data an effect would destroy, and *capturing* it means saving it before the command runs, by the cheapest means that suffices — a hardlink, a copy, or a record — so that undo can restore it.

Two mechanisms find a command's effects. *Composition* follows effects through the operators that join commands into one line (`;`, `&&`, `||`, pipes, and redirection) and through the commands that run other commands (wrappers such as `sudo`, `find -exec`, `xargs`, `sh -c` strings, `eval`, and loops). *Resolution* learns targets known only at run time by running a form of the command first proven to change nothing. From those effects the analysis reaches a *verdict*, one of three: *recoverable*, if undoing the command needs nothing saved because it only creates or moves files; *recoverable with capture*, if undoing it needs pre-images saved first; and *unrecoverable*, if no capture would restore what it does, or the analysis cannot tell. The analysis *fails closed*: anything it cannot determine is unrecoverable, so the user is asked. Which verdicts are put to the user is set by a confirmation *policy*: the *strict* policy asks before every command that is not plainly recoverable, and the *capture* policy asks only before unrecoverable ones, capturing the rest silently.

Because the analysis takes a command as a string, it is independent of the interface and of the model that wrote the command. The interface used here, SynapseOS, is a conversational interface layer built for this study over a Linux desktop that keeps running underneath it, and is the experimental platform on which the analysis is validated with people, against each participant's own primary operating system. The setting has

gaps of its own (Chapter 2, Section 2.9): no surveyed system, implemented and evaluated, places a conversational layer over a whole desktop session for the person using the machine [25, 54]; the Linux work that exists serves servers and agents, not an end user's desktop [1, 13]; and controlled comparisons with the conventional graphical workflow cover narrow slices, such as a single application [26]. These gaps are not this study's contribution, but they are why the analysis is validated with people in a working interface and not only on a corpus.

1.3 Statement of the Problem

Language models have made it practical to turn a request in ordinary language into a shell command, and conversational interfaces, coding assistants, and agentic tools increasingly run what a model writes on the user's own system. A generated command can be wrong, or broader than the request intended, and the tools that run such commands today do not determine what must be saved for its damage to be undone (Section 1.2). This study addresses the problem of determining, before a generated shell command runs, what it would change, whether the change can be undone, and the least that must be captured beforehand so that it can be. The problem is studied where it is acute — a person who does not read shell syntax, operating a Linux desktop through conversation — but the analysis applies wherever generated Bash commands run on a system like the one studied.

Specifically, the study seeks to answer the following questions:

1. How can the effect of a shell command on a system be represented formally, so that whether the command can be undone is a predicate over that representation, and so that anything the representation cannot determine is treated as unrecoverable rather than safe?
2. Can a command's effects be derived from its syntactic structure before it runs — composed through the operators that join commands and the commands that run other commands, with targets known only at run time (command substitutions,

`find` matches, `xargs` input, archive members) resolved by commands first proven read-only — without executing anything that changes the system?

3. Given those effects, can a minimum-cost set of pre-images be computed whose capture is sufficient to restore the prior state, and does restoring from it return the system to that state?
4. Against an enumerated list of destructive patterns, a fail-closed list that asks before every command it does not recognize as read-only, and the command-safety checks of open-source AI coding agents, does the analysis lose less data silently, restore more of what is lost, and ask the user less often?
5. How much of that result do composition and runtime-target resolution each contribute, beyond the per-command rule table on which both rest?

A further question is answered by validation rather than posed as a research question: whether the analysis's decisions hold when it is deployed in a working interface and exercised by people, whose commands the evaluation corpus does not contain (Chapter 3, Sections 3.3–3.7).

1.4 Objectives of the Study

The general objective of this study is to design and evaluate a recoverability analysis for shell commands generated by a language model: an algorithm that determines, before a command runs, what it would change, and computes the minimum-cost set of pre-images that must be captured for the command to be undone. The algorithm is demonstrated and validated in use in SynapseOS, the experimental platform.

Specifically, this study aims:

1. To formalize an effect-semantics model for shell-command recoverability: the kinds of effect a command can have on the system, a recoverability verdict over them, and a one-sided soundness rule under which anything that cannot be determined is treated as unrecoverable (RQ1).

2. To develop an effect-composition algorithm that derives a command's effects from its syntactic structure rather than its name, composing them through the commands that run other commands and resolving targets known only at run time through dry runs it first proves read-only (RQ2).
3. To develop a minimum-cost recovery-plan generation algorithm that computes, from a command's effects, the cheapest set of captures sufficient to restore the prior state, and to verify by sandboxed execution that restoring from that plan does so (RQ3).
4. To evaluate the recoverability analysis against pattern-based and fail-closed baselines and the command-safety checks of open-source AI coding agents, on a held-out command corpus whose ground truth is obtained by sandboxed execution, measuring silent data loss, verified restoration, and needless confirmation, with statistical testing (RQ4).
5. To quantify the contribution of composition and of runtime-target resolution through ablation studies that disable each, and both, while leaving the per-command rule table unchanged (RQ5).

Each objective is algorithmic and measurable, and each would stand if the interface were removed. SynapseOS is therefore the platform on which the analysis is validated in use (Chapter 3, Sections 3.3–3.7), not an objective in itself.

1.5 Contributions of the Study

This study contributes one thing, an algorithm, and builds only what that algorithm needs to be deployed and tested. It follows one design principle, which shows where its contribution lies: adopt existing software wherever it will serve, and build only what cannot be adopted. Most of what is adopted, listed in Table 1.1, is the platform's stack; what is built is the analysis, whose novelty against prior work is argued in Chapter 2.

Table 1.1. Components adopted rather than built, and why.

Concern	Adopted	Why it is not built here
Kernel, init, system services, userland	Debian 13 (Trixie)	Writing these is operating-system development in the strict sense and is explicitly outside this study
Desktop environment, window manager, display server	XFCE on X11	The interface layer runs over the desktop and leaves it working; reimplementing it would remove capability from the user
Model inference and serving	Ollama	Model lifecycle, quantized loading, and request serving are solved and are not the research question
Language model	Qwen2.5-Coder-3B-Instruct on the smallest machines; a larger local model where memory allows	Training a model is a different study; the analysis this study contributes is independent of the model that produced the command
Command execution	The Linux shell and core utilities	Driving the existing toolchain is the system's premise, not a limitation of it
Version-control recovery	Git's object store	For files a repository tracks, one captured commit identifier restores their committed state, where copying each file would cost far more
Package and service state	apt, dpkg, and systemd's own queries	The analysis asks the system's own tools what a command would do and derives its inverse, rather than re-implementing them
Sandbox for resolving targets	bubblewrap	Mature, unprivileged, and packaged for Debian; it runs the analysis's read-only resolvers with the whole filesystem mounted read-only

Beyond these adopted components, the study makes one research contribution and builds two supporting artifacts that serve it. The research contribution is the following.

1. **An algorithm for recoverability analysis of generated shell commands.** The algorithm derives a command's effects from its structure, through composition and resolution; decides whether the command can be undone; and computes the minimum-cost set of pre-images to capture before it runs. Its formal basis is an *effect semantics* in the tradition of abstract interpretation, which reasons soundly about what a program could do without running it, and of type-and-effect systems, which track what a program changes and not only what it computes [8, 20, 44]. A command is represented by the set of effects its syntax implies, resolved against a working directory, and the verdict is a predicate over that set. Soundness is one-sided: over-approximating what a command destroys costs a confirmation, whereas under-approximating it destroys data. The analysis rests on a table of what each known command does, and a command absent from it is unknown and asks; the claim is what the analysis adds on top of a table, not that it has none. A permission model can refuse a command but cannot restore what an approved one destroyed; the analysis instead captures beforehand what it determines a command would destroy, and unlike *sudo* and *polkit*, it gates on what is about to happen rather than on who is acting. This algorithm is the study's principal technical contribution and the subject of RQ1 through RQ5.

   It was evaluated on a held-out corpus of 777 commands, drawn after the analysis was frozen from NL2Bash, a public dataset of English descriptions paired with Bash commands, and from templates and commands written for the purpose. For the 697 that act on files, ground truth was observed by running each in a sandbox; the 80 whose effects lie outside the filesystem, such as on another machine, were labeled from a fixed table of state-changing tools. On this corpus, the analysis let none of the 405 commands that destroy data or act outside the filesystem run without asking first or, for those that act on files, capturing first, where an enumerated pattern list let 41.2% through. Of the 325 that destroy files, the pattern list let 31% through and the shipped checks of Codex CLI and cc-safety-net 73% to 97%, though those checks ask about fewer

harmless commands. A fail-closed list, which asks before every command it does not recognize as read-only, ties the analysis at zero silent loss, so the analysis's gain is what it restores and how much less often it asks. Its *capture coverage*, the share of the 325 it captured beforehand and undo then restored exactly, was 89.5%, where the fail-closed list restored none. Of the 372 commands that lose nothing, it asked about 9% to 12%, depending on the policy, against the fail-closed list's 27%, and not significantly more often than the pattern list.

Ablations, which rerun the analysis with one part switched off and the same table kept, show where the result comes from: capture coverage falls to 35.4% without composition, 45.5% without resolution, and 32.0% without both. Each drop is significant under McNemar's exact test, which compares two systems on the same commands (each $p < 10^{-42}$), and silent loss stays at zero throughout. Chapter 3, Sections 3.1–3.2, reports the full results.

The two supporting artifacts are not research contributions; they are how the contribution is deployed and validated in use.

1. **SynapseOS, the experimental platform.** A single binary running over an unmodified desktop session, it grounds natural-language intent in shell commands, gates each command on the analysis's verdict, captures what the analysis's plan names before the command runs, and records a recovery journal from which the command can be undone.
2. **A validation of the analysis in use.** A controlled study in which 40 participants, spanning novice users to power users and measured by a proficiency score rather than sorted into groups, operate the platform on matched tasks. It measures silent loss, verified restoration, and needless confirmation on commands people cause, and characterizes the platform through task success, time, error rate, and workload against each participant's own primary operating system.

The research contribution is evaluated on a labeled command corpus, independently of any participant, so it is reported separately from the platform and its validation, and a negative result on any of the three is reported as such.

1.6 Significance of the Study

The analysis matters to anyone who builds, studies, or relies on systems that run commands a language model writes, and its findings are expected to benefit the following groups:

1. **Developers of AI agents that run shell commands.** Coding agents guard generated commands with lists of dangerous patterns. On this study's corpus, the shipped checks of Codex CLI and cc-safety-net let most file-destroying commands run with neither a confirmation nor a capture, and even this study's pattern-list baseline let some through (Section 1.5) [21, 32]. The analysis adds what a list cannot, a capture from which the command can be undone, and can sit in front of any system that runs generated Bash commands on a Debian-like Linux system, the setting studied here (Section 1.7).
2. **The research community.** The study contributes a static analysis of shell commands for recoverability, a property the surveyed analyses of shell programs do not address (Chapter 2, Section 2.5). It is evaluated against ground truth observed by sandboxed execution for every command that acts on files, not assigned by human labelers, with ablations that separate what composition and runtime-target resolution each contribute from the rule table beneath them.
3. **People who direct a computer through conversation.** A person who cannot read a shell command cannot judge one a model wrote for them, so letting a model act for them is safe only if what it destroys can be restored; the analysis supplies that for the commands it can determine, and asks first about the rest. The validation study also characterizes the platform for novice and power users,

and the platform could later host accessibility-specific configurations, such as speech-driven operation for users with motor impairments [14, 33, 34], which are outside this study's scope.

1.7 Scope and Limitations of the Study

This study covers the design and evaluation of a recoverability analysis for generated shell commands, a program analysis that is its research contribution; it draws on natural language processing, LLM-based reasoning, Linux system integration, and human-computer interaction for the setting in which the analysis is deployed. The analysis covers commands in the Bash dialect on a Debian system: their effects on the filesystem, and their changes to package and service state through apt, dpkg, and systemd. It is evaluated and ablated on the 777-command corpus of Section 1.5, whose templates cover each composition operator.

The study also covers the platform in which the analysis is deployed. SynapseOS is a *conversational interface layer*, the surface through which intent is expressed and results are returned, and it sits above the *session layer*[3] of the Linux desktop[4]. It runs as an agentic layer over an ordinary desktop session rather than in place of one, so the desktop shell, session manager, and application launcher remain available to the user, and it acts on the user's request by translating it into shell commands run through the standard command-line toolchain. The Linux kernel, the core system utilities, and the existing ecosystem of Linux applications are not modified. The validation study, bounded by the duration of the thesis project, recruits participants from novice computer users to power users for cross-application desktop workflows that include shell operations, file management, and tasks representable in the OSWorld benchmark task suite [50].

The following limitations are acknowledged, those of the analysis first.

First, what the analysis cannot determine, such as code run by an interpreter, programs it does not model, and variables with no value, it treats as unrecoverable, and so protects only by asking first. Of the 325 file-destroying commands in the corpus, 34

(10.5%) are protected only this way (Chapter 3, Section 3.1(d), lists them by cause).

Second, parts of the analysis rest on weaker evidence or on assumptions. Its model of package and service changes is checked by unit tests rather than by execution, since no command in the corpus changes installed packages or running services under observation (Chapter 3, Section 3.1(e)). Its per-command rules are a hand-written enumeration, which composition builds on rather than replaces. It resolves targets shortly before the command runs, so a file another process changes in the interval is captured as it was when resolved. Undo restored 630 of the 631 commands executed in the recovery check, and testing after the evaluation found seven defects in the analysis and one in its undo, all fixed and none triggered by any command in the corpus, so re-running the evaluation changed no result (Chapter 3, Section 3.1(d), reports the failure and the defects).

Third, the corpus bounds what the evaluation shows. It combines human-written commands from NL2Bash with templates written by the same team that wrote the analysis's rule table and the pattern-list baseline. Each label describes what a command did on its *fixture*, the files and directories prepared for it to run on, not on every possible filesystem; and the 80 external commands, a fifth of the 405 behind the silent-loss result, are labeled from a table, weaker evidence than the 697 observed in the sandbox. The corpus was also written by people, not by the model. On a smaller partition of commands the deployed model wrote, the result pointed the same way, but the analysis let one loss through, on a command that partition's own labeling rule mislabeled, and its advantage in capture coverage over the pattern list was not significant; that result rests on 19 losses (Chapter 3, Section 3.1(d)). Chapter 3, Section 3.1(e), states these threats and their mitigations.

Fourth, the recovery plan is minimum-cost relative to the capture mechanisms and cost model stated in Chapter 3, Section 2.1(c), not over every conceivable way of restoring a system.

Fifth, the analysis does not ask less often than the pattern list, which it does not significantly exceed (32 to 45 against 30 of 372), or than the coding-agent checks (1 to 15). The deployed gate, which also consults the pattern list, asks about 58 to 67 (16% to 18%), still well below the fail-closed list's 27%. How often the user is asked also depends on the policy, strict by default or capture (Chapter 3, Section 2.1(e)), and which serves users better the validation study informs rather than settles.

Sixth, the platform commits to a locally hosted, open-weights, code-specialized language model sized to the machine (Qwen2.5-Coder-3B-Instruct on the smallest machines) rather than a hosted frontier model, prioritizing privacy and offline operation over the potentially higher capability of larger hosted alternatives[5]. This may limit how often a request is translated correctly, which bounds the validation study but not the analysis, since the analysis takes whatever command it is given.

Seventh, the platform targets the Linux desktop, and the validation study's findings may not generalize to Windows or macOS. The study is also bounded by the task coverage of the OSWorld benchmark and the custom task suite, and it excludes users with motor accessibility needs for ethics-review and recruitment reasons, so accessibility-specific configurations such as speech-instructed operation for atypical speech [14] are out of scope.

Notes

1. Foundation models are large neural networks trained on broad, general-purpose data and adaptable to many tasks. Multimodal variants accept multiple input types — typically text and images — enabling the model to interpret visual interfaces and screenshots alongside natural language.
2. Computer-using agents are AI systems that interact with a computer's graphical environment — clicking, typing, reading the screen — to complete tasks on behalf of a user. Unlike shell-level automation, they operate through the same visual interface a human would use.
3. The session layer (or simply, the session) is the software the user directly interacts with immediately after login — the desktop shell, window manager, session manager, and application launcher on a graphical system, or the login shell on a text terminal. It is distinct from the userland (or user space): the core system libraries and command-line tools (e.g., glibc, coreutils,

bash, apt) that run outside the kernel and that the session layer itself depends on and invokes. SynapseOS sits above the session layer and does not replace it; the underlying userland is unchanged.

4. A Linux session manager is the program that starts after user login and establishes the graphical environment — launching the window manager, desktop shell, and background services. SynapseOS does not replace this component: it launches over the session that the component establishes and becomes the surface the user works in.
5. Open-weights models make their trained parameters publicly available for local download and use. Users run inference on their own hardware without routing queries to an external service provider, enabling offline operation and preventing user data from being transmitted to third parties.

Chapter 2

REVIEW OF RELATED LITERATURE

2.1 Introduction to the Review

This chapter reviews two bodies of research. The first is the program-analysis and command-safety work on which the study's recoverability analysis builds; the second is the work on conversational and language-model-mediated interfaces to personal computing, in which SynapseOS, the platform where the analysis is deployed, is situated. The review spans nearly four decades of work on natural-language interfaces to computing systems, from the symbolic-artificial-intelligence consultants developed for the Unix shell in the late 1980s [49] to the large language model (LLM) agents that now operate shells, graphical applications, browsers, and mobile devices. It has three purposes: to establish the conceptual vocabulary and technical foundations on which the present study builds, to characterize the current state of the field and its recurring limitations, and to locate precisely the gaps that the proposed recoverability analysis, and the system in which it is deployed, are designed to fill.

Works were included in this review if they satisfy at least one of seven criteria: they analyze or guard shell commands or scripts, reason statically about a program's effects, or restore a system's state after a change; they translate natural language into executable system actions; they operate graphical, web, or mobile interfaces through LLM-based reasoning; they address speech- or accessibility-driven interaction with computing systems; they provide an evaluation environment for such systems; they report a controlled empirical comparison between a language-mediated interface and a conventional graphical one; or they set out the interaction theory, and the critiques of natural-language interaction, against which such an interface must be judged. Works

concerned solely with model architecture or training, without a computing-interaction target, and general-purpose conversational systems without a system-control capability, were excluded unless they bear on that interaction theory. The comparison criterion is stated explicitly because the fourth gap this chapter identifies concerns the scarcity of exactly such comparisons. A corpus that admitted only the other criteria could not contain a comparative interface study by construction, and an absence argument drawn from it would be circular rather than evidential.

Candidate works were identified through language-model-assisted search across arXiv, the ACM Digital Library, IEEE Xplore, and the ACL Anthology, and the search was extended by backward and forward citation chaining from each work that met the criteria above. Because shipped systems that guard or recover from agents' commands are documented outside those databases, vendor documentation and open-source repositories were also searched directly for command guards and recovery mechanisms, as of October 2026; ANOLISA, Codex CLI, and cc-safety-net were found this way. The review is therefore narrative rather than systematic, and the search is not claimed to be exhaustive. Assisted search can also surface plausible but non-existent references, and can attribute to a source a claim that source does not make, so every work cited in this chapter was subsequently verified by hand against its primary record — author list, venue, and year — and, where a specific claim is attributed to a work, against the presence of that claim in the work's own text. That verification pass corrected errors of both attribution and currency introduced by the initial search, and it is reported here because the reliability of this chapter rests on it rather than on the discovery method. The limitation that remains is one of recall rather than accuracy: assisted search favors well-indexed and heavily cited work, so the corpus may under-represent less visible literature, and the absence claims in §2.9 are accordingly scoped to the surveyed corpus and stated as such, rather than as claims about the literature as a whole. With its scope and method set out, the chapter first presents the theoretical frameworks (§2.2) and then proceeds thematically — historical foundations and the case against natural-language interaction (§2.3), natural-language-to-shell translation (§2.4), command safety, static analysis of

shell commands, and recovery mechanisms, the literature closest to the study's algorithm (§2.5), graphical and computer-using agents (§2.6), accessibility and speech-driven interaction (§2.7), and evaluation environments (§2.8) — before synthesizing the surveyed works into a statement of the research gap and a comparative analysis (§2.9). It closes with a review of the methodological literature that informs the validation study (§2.10) and a summary that motivates the methodology detailed in Chapter 3 (§2.11).

2.2 Theoretical Framework

This review rests on two frameworks of different standing. The first is the theoretical basis of the study's contribution: the program-analysis tradition of which the recoverability analysis is an instance. The second organizes the literature on language-mediated interfaces, in which the platform that deploys the analysis is positioned.

Program analysis: abstract interpretation and effect systems

The technique on which the study's recoverability analysis relies — determining what a piece of code will do without executing it — is not new. It has a decades-long history in programming-language theory under two related names, neither of which the natural-language shells and command guards surveyed in this chapter draw on. Abstract interpretation, introduced by Cousot and Cousot [8], is the foundational technique for computing sound, conservative approximations of a program's runtime behavior by evaluating it over an abstract domain rather than its concrete one. Type-and-effect systems extend a program's type judgment with an explicit account of the side effects an expression may produce, so that a compiler or static checker can reason about those effects instead of leaving them implicit in execution; their types and effects can be inferred by the algebraic reconstruction of Jouvelot and Gifford [20], and Talpin and Jouvelot [44] give them a full formal treatment. The recoverability algorithm this study contributes (Chapter 3, Section 2.1(a)–(c)) is an instance of this discipline, applied to a question to which no work in the surveyed corpus applies either tradition: whether the effects of a generated command can be undone, and what must be captured beforehand so

that they can be. The algorithm treats a generated shell command as an expression whose effect is derived from its syntax and composed across its operators — an effect on the filesystem and on the system's installed packages and running services, rather than on a type system's abstract state — and it computes the recoverability verdict over that effect set, not over the command's name. This connects the algorithm to an established formal lineage rather than presenting it as a bespoke heuristic. It also grounds the algorithm's one-sided soundness property — over-approximating destructiveness under uncertainty rather than under-approximating it — in the same requirement abstract interpretation itself imposes: a sound abstraction may lose precision, but it may never assert a property the concrete execution does not actually have.

Two parts of the analysis go beyond a purely static reading. The first concerns targets known only at run time: for these, the analysis runs a form of the command first, which makes it a hybrid of static and dynamic analysis. Its soundness still rests on the static part, because a form is run only when the static analysis proves it can change nothing, and a target that cannot be resolved this way falls back to unrecoverable. The second is the recovery plan, a cost-minimizing choice over that abstraction: for each effect the abstraction admits, it selects the cheapest capture that restores it, so the plan inherits the abstraction's over-approximation, capturing more than strictly necessary when the analysis is unsure, never less. Combining static and dynamic analysis is not new in general; what is specific to this study is the combination it uses: dry runs run only after the static analysis proves them read-only, feeding a recoverability verdict, and a recovery plan chosen over that verdict's effects.

Three properties of the analysis follow from this framework and are carried through the rest of the study. Its abstract domain is the *effect set*: what a command would read, create, write, remove, or alter in metadata, or change in the system's installed packages and running services, not the command's name alone. Its semantics is *compositional*: the effects of a line are built from the effects of its parts through the operators that join them, which is what an enumerated list of command patterns cannot express. And its soundness is *one-sided*: whatever cannot be determined is treated as

unrecoverable, so that the analysis may ask more often than necessary but may never declare a destructive command safe, a guarantee that holds relative to the per-command rule table and to the filesystem as it is when targets are resolved, shortly before the command runs (Chapter 3, Section 2.1(a)).

The four-layer interaction pipeline

The second framework organizes the interface literature. The heterogeneous systems surveyed in this chapter — shell translators, web and mobile agents, desktop automation frameworks, and accessibility interfaces — differ widely in platform, action space, and evaluation method, yet they address a common problem: converting a user's expressed intent into effects on a computing system. To compare them on common ground, this review adopts a *four-layer interaction pipeline* of Input, Reasoning, Action, and Operating-System Integration. The framework is informed by the component view of large language model (LLM)-brained graphical user interface (GUI) agents in the survey of Zhang and colleagues [53]; its perception, planning, and action components are recast here as four layers for language-mediated computing interfaces as a class, with an added layer for the scope of an interface's effect. Figure 2.1 depicts the pipeline and situates the layers relative to one another.

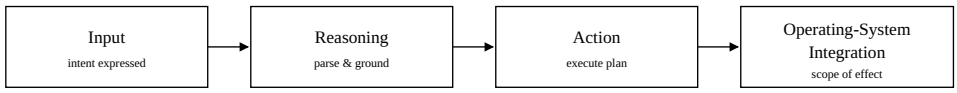


Figure 2.1. The four-layer interaction pipeline for language-mediated computing interfaces, adapted from the LLM-brained GUI-agent survey of Zhang and colleagues [53]. A user's intent enters at the Input layer, is parsed and grounded at the Reasoning layer, executed at the Action layer, and takes effect within a scope defined at the Operating-System Integration layer.

The *Input* layer concerns how a user expresses intent, whether by typed text, speech, or structured shortcuts. The *Reasoning* layer concerns how that intent is parsed into a plan and grounded into the affordances of the environment. The *Action* layer concerns how a grounded plan is executed — through command translation, through binding to native application programming interfaces (APIs), or through simulated keyboard and mouse operations. The *Operating-System Integration* layer concerns the scope of computing an approach can affect, from a single application to an entire operating system.

This framework suits that literature for three reasons. First, it is *modality-neutral*: because it abstracts over whether intent arrives as text, speech, or gesture and whether action is realized through shell commands, API calls, or simulated clicks, it accommodates the full span of surveyed systems without privileging any one interaction style, a necessary property for a review that spans command-line, graphical, and accessibility work. Second, it is *diagnostic*: locating each system's principal limitation at a specific layer — visual grounding as a Reasoning-and-Action failure, or narrow platform coverage as an Operating-System Integration limitation — turns an otherwise flat catalogue of systems into a structured comparison, and the comparative synthesis of §2.9 makes this layer-by-layer positioning explicit. Third, it is *directly applicable* to the platform. SynapseOS, in which the analysis is deployed (Chapter 3, Figure 3.1), instantiates the same pipeline: it accepts natural-language intent at the Input layer, parses it with a locally-hosted small language model at the Reasoning layer, translates it into shell commands executed through the standard toolchain at the Action layer, and spans the full graphical session rather than a single application at the Operating-System Integration layer. The pipeline therefore serves both as the organizing lens for the interface literature and as the vocabulary in which the platform is later positioned. Within it, the analysis sits between the Reasoning and Action layers, where a generated command is about to take effect.

2.3 Historical Foundations of Natural-Language Interfaces

The aspiration to let users describe what they want in ordinary language and have a computer carry it out is not new: an early system of this kind for an operating system, the Berkeley Unix Consultant of Wilensky and colleagues [49], was already operational by the late 1980s. It was an intelligent natural-language interface that allowed naive users to learn about the Unix operating system through conversation in ordinary English, addressing both the natural-language-frontend problem and the consultation problem, and so distinguishing itself from systems intended only as command translators. Built on the KODIAK knowledge representation, it incorporated a user-modeling component that represented the user's knowledge state and a teaching component through which users could extend its knowledge base in natural language. The system demonstrated end-to-end natural-language consultation about Unix, successfully answering "how do I," definitional, and problem-solving questions. Critically, its authors identified knowledge acquisition as the principal bottleneck: extending the system to a new domain required substantial hand-engineering, a concern that directly foreshadows the contemporary problem of LLM-based agents requiring per-application grounding data.

The intervening decades saw natural-language interaction treated primarily as a supervised-learning problem. Representative of this pre-LLM era, the work of Li and colleagues on mapping natural-language instructions to mobile user-interface action sequences [23] trained dedicated phrase-extraction and grounding transformers on hand-annotated action sequences, achieving 70.59% accuracy on predicting complete ground-truth action sequences on its benchmark but remaining bound to its purpose-built datasets and architectures. The decisive shift came with the introduction of in-context reasoning patterns for LLMs. ReAct, introduced by Yao and colleagues [51], collapsed the multi-stage supervised pipeline into a prompting strategy: the language model is prompted to alternate between free-text "Thought" steps that update its plan and "Action" steps that interact with an external environment, with observations fed back into subsequent

reasoning. Implemented purely as a prompting strategy without fine-tuning, ReAct nonetheless outperformed imitation-learning and reinforcement-learning baselines that required thousands of training trajectories.

The historical significance of this transition is twofold, and it frames the remainder of this review. First, it explains why progress in the field has been so rapid since 2023: the methodological foundations became portable across modalities and platforms in a way that the earlier dataset-bound approaches were not. Nearly every system surveyed in the sections that follow inherits from ReAct the convention of interleaving reasoning traces with environment-affecting actions. Second, this shared inheritance underscores a corresponding fragility: these systems inherit the failure modes of LLM reasoning — hallucinated actions, plan inconsistency, and brittleness on long-horizon tasks — alongside their capabilities, a concern that the evaluation literature reviewed in §2.8 quantifies.

Interface paradigms and the case against natural language

The aspiration to a natural-language interface has a long-standing critique that remains unanswered, and a review that omitted it would misrepresent the state of the question. Shneiderman's account of direct manipulation [41] holds that interfaces succeed when the objects of interest are continuously visible, when actions upon them are rapid, incremental, and reversible, and when their effects are immediately apparent — properties that let a user build and maintain an accurate model of system state. Norman formalized what an interface does for a user as the bridging of two gulfs [31]: the gulf of execution, between an intention and the actions a system makes available, and the gulf of evaluation, between the system's response and the user's ability to interpret it. Hutchins, Hollan, and Norman [18] argued that direct manipulation narrows both, by making an interface feel like the object of interest itself rather than a description of it.

A language interface sits awkwardly against nearly every one of those properties. An utterance is not continuously visible, not incremental, and not reversible by the same gesture that produced it; the set of available actions is not displayed, so the

user must guess at the system's capabilities rather than see them. Shneiderman and Maes debated precisely this trade-off between direct manipulation and interface agents [43], and Shneiderman argued subsequently that speech and language interaction consume the same cognitive resources a user needs in order to hold a task in mind, where pointing does not [42]. Empirical work on modern conversational agents has largely borne the concern out: Luger and Sellen documented a persistent gulf between what users expected a conversational agent to understand and what it actually did [27], and Clark and colleagues found that the properties people value in human conversation transfer poorly to interaction with an agent [7].

This critique is the strongest argument against the interface premise of the platform, and it shapes the platform's design rather than being set aside by it. Two of its three concerns are addressed directly. The gulf of evaluation is narrowed by displaying the generated command and its raw output rather than reporting an outcome in prose, so that the user sees what was actually done rather than a description of it. The irreversibility of an utterance is answered by the confirmation gate and the undo journal, which restore by mechanism the reversibility that direct manipulation obtains for free. The third concern — that a user cannot see the boundary of what the system is able to do — is not solved by this design and is carried forward as an acknowledged limitation.

Beyond the critique lies the comparative question, which is older than the current literature and considerably less settled than it is often assumed to be. It is widely held that the command line's power is inaccessible to most people in practice, yet the best-known controlled comparison of command, menu, and iconic interfaces, conducted by Whiteside and colleagues across seventy-six users and seven systems [48], concluded that interface style itself predicted neither performance nor preference: careful design did, and there was no necessary tradeoff between ease of use and ease of learning. That finding is as much a caution for the present study as a motivation for it. It warns against attributing to a conversational paradigm what may in fact belong to the quality of one

particular implementation, and it is a principal reason this study measures a specific system against a specific baseline rather than claiming to measure one paradigm against another.

2.4 Natural-Language-to-Shell Translation

The thread of the literature most directly related to the platform concerns the translation of natural language into shell commands. Because SynapseOS grounds user intent into commands executed through the standard Linux command-line toolchain, the accuracy, safety, and context-awareness of natural-language-to-Bash translation are of central relevance to its design.

The standard dataset for this task is NL2Bash, from Lin and colleagues [24]: over nine thousand English descriptions paired with Bash commands, collected from question-and-answer sites, tutorials, and course material. It supplies the human-written commands of this study's evaluation corpus (Chapter 3, Section 1.2(a)). Because its commands were collected for translation rather than for safety, most of them are read-only, and that corpus is therefore sized against the labels it must contain rather than drawn as the dataset comes.

Westenfelder and colleagues [47] address this translation problem directly. Their work constructs a manually verified test set of 600 instruction–command pairs and a training set of 40,939 pairs, and it introduces a functional-equivalence heuristic that reaches 95% confidence in determining whether two Bash commands produce the same effect, allowing automated evaluation to move beyond exact-string matching to a measure of whether a generated command actually accomplishes the intended task. Their fine-tuned models substantially outperform zero-shot baselines, demonstrating that fine-tuning specifically on command-translation data is effective. The authors identify two persistent limitations, ambiguity in user instructions and the absence of system context at translation time — the contents of the current directory, the user's command history, and

environment variables — and integrating context-aware retrieval is a direction their findings point to, one that the persistent conversational session of SynapseOS is well positioned to supply.

The feasibility of deploying such a system without reliance on a hosted frontier model is supported by the large-scale evaluation of Jacobs and colleagues [19], who assess a broad range of locally deployable open-source LLMs on Bash command generation. Their work provides an empirical anchor for the viability of running natural-language-to-Bash translation on local, open-weights models of modest size, the deployment posture adopted by SynapseOS for reasons of privacy and offline operation. Taken together, the shell-and-command-line thread establishes that natural-language-to-Bash translation is tractable, that safety and reversibility are central rather than incidental concerns, and that local deployment is feasible. Yet no surveyed shell-level system has been evaluated with users against the conventional workflow it would supplement. How such systems guard the commands they generate, and what static analysis of shell commands can establish, are taken up in §2.5.

2.5 Command Safety, Static Analysis, and Recovery

The literature closest to the study's algorithm concerns how shell commands are checked before they run. It falls into three groups: *guards*, which decide whether a generated command may run; *analyses*, which reason about what a shell program does; and *recovery mechanisms*, which restore a system after the fact. Each supplies part of what the study needs. None, as this section shows, determines before a command runs whether its effects can be undone and captures exactly what undoing them requires.

Guards

The most direct response to the risk of a generated command is a guard, which decides whether the command may run at all. Where Westenfelder and colleagues study translation accuracy (§2.4), Gyawali and colleagues study safety. Their NaSh system [13], published under the operating-systems classification, proposes guardrails for an

LLM-powered natural-language shell for Linux: the model proposes a candidate command, a guardrail layer between it and the user evaluates the command against safety, reversibility, and confirmation criteria, and the user receives a structured preview before execution. Because LLM outputs may be unintended or unexplainable, the authors note, guardrails are required for user error recovery; they leave two problems open: recovery and rollback mechanisms for irreversible system operations, and empirical user evaluation. The study's recoverability analysis addresses the first, recovery from an operation already approved; its validation in use, the second.

Shipped tools follow the same design. Coding agents guard the commands they execute with lists of command patterns: the dangerous-command check of OpenAI's Codex CLI [32] and cc-safety-net [21], a pre-execution guard used with several agentic coding tools, are representative open-source examples, and both are scored as baselines in Chapter 3, Section 3.1(a). Both assume a version-controlled project, where deleting project files is routine and version control, not the guard, makes a mistake recoverable. Files on a personal computer, the setting of this study, have no such safeguard.

Every shipped guard surveyed decides by *permission*: allow-and-deny lists over command prefixes match the command against an enumerated set of patterns judged dangerous, and the command is then allowed, refused, or escalated to the user. NaSh [13] goes further as a proposal, adding summaries of what a command would affect and selective undo, but leaves the mechanisms for both open. Permission answers whether a command may run, not whether it can be undone once it has, and the two answers differ. Refusal protects a user only until they approve, which, for any system that must let work proceed, is most of the time. This gap, which the guards share, is the one the study's algorithmic contribution addresses.

The distinction matters because enumeration does not compose. A pattern list matches a command in isolation, but a generated line routinely chains operations with different recovery requirements: a removal, a version-control reset, and an in-place edit joined by &&, with redirects and command substitution changing what each actually touches. No enumeration of command prefixes expresses that, and extending the list

does not converge: each newly named tool leaves the next one unnamed. The comparison that matters, though, is not only with a list of dangerous commands. Turned around to ask before every command it does not recognize as read-only, a list closes the gap left by unnamed commands at the cost of asking about everything unfamiliar; this fail-closed list is the strongest simple baseline. The analysis does not escape enumeration either: its effects come from a table of what each known command does, and a command the table does not name is treated exactly as the fail-closed list treats it, by asking. What it adds is composition over the table, read-only resolution of run-time targets, and the capture that lets a command run and still be undone. The study therefore claims what the analysis adds to the fail-closed list, not the verdict alone: capturing what would be lost so that undo restores it, and asking far less often, both made possible by composition and resolution. Determining recoverability over a composed command, and computing the minimal information needed to reverse it, is the open problem this study takes up (RQ1–RQ3), and it separates the present work from the guardrail designs surveyed above.

Risk in tool-executing agents has also been studied by evaluation rather than guarding. ToolEmu emulates tool execution with a language model so that agents' failures, including destructive actions, can be found before deployment [38]. It measures how risky an agent is across many cases; it does not protect a user from the particular command an agent is about to run.

Static analysis of shell programs

Static analysis of shell scripts is comparatively little studied, and existing work targets properties adjacent to recoverability rather than recoverability itself. ABash [28] performs abstract interpretation over bash's variable-expansion semantics to catch common expansion bugs, with taint tracking, but it does not consider what a script's commands would do to the filesystem if executed correctly. Smoosh [12] gives the POSIX shell a mechanized, executable small-step operational semantics which, run against the POSIX and Modernish test suites and its authors' own, proved more conformant to the standard than seven widely used shells: exactly the kind of ground-

truth semantics against which a static effect analysis would need to be checked, though Smoosh defines the semantics rather than analyzing programs written in it. Closest is CoLiS [2], which applies symbolic execution to the more than twenty-seven thousand POSIX-shell maintainer scripts in the Debian package archive, scripts that run as root during install, upgrade, and removal, to find violations of properties Debian policy requires, among them *idempotence*: running a script a second time causes no further harm. Idempotence and recoverability are related but distinct. For example, *rm -rf /var/cache/pkg* is idempotent, since a second run does no more damage, and destructive on its first run, since nothing restores what it removes unless that was captured beforehand; conversely, a command can be recoverable without being idempotent, if each run produces a new, independently reversible effect. CoLiS, the nearest prior application of static effect reasoning to shell scripts, therefore stops short of the present question: verifying that re-execution is safe does not determine whether the first execution can be undone. No work identified in this review derives a recoverability verdict for a shell command from its statically computed effects; that is the specific gap the study's algorithmic contribution fills.

Recovery after the fact

Guards and analyses act before a command runs. A separate literature restores a system after something has gone wrong, asking not what a command will destroy but how to recover what was destroyed. Three kinds of mechanism are relevant.

Filesystem snapshots are the most widely deployed, and one shipped natural-language shell relies on them: ANOLISA's checkpoint tool snapshots an agent's workspace at file level before key operations or after each round of conversation, and rolls back to any snapshot [1]. A copy-on-write filesystem such as btrfs snapshots a whole subvolume cheaply, because a snapshot shares its blocks with the live filesystem until they are modified [37], so snapshots can be taken on a schedule or before system updates; without copy-on-write, as on the ext4 on which this study is conducted, a snapshot must visit every file in the tree, a cost measured in Chapter 3, Section 3.1(a). A

snapshot needs no knowledge of the command it protects against, which is both its strength and its limit: restoring one returns the whole subvolume to an earlier moment, discarding every later change along with the unwanted one, and it cannot tell the user beforehand whether a command will lose anything. Versioning file systems, the second kind, keep old versions of individual files: the Elephant file system retains them under per-file policies that decide when an old version may be forgotten [39]. Recovery is then per file and needs no foresight, but storage is paid for on every change, mistaken or not, and obtaining it means replacing the filesystem.

The third kind, undo at the level of a service or the operating system, makes recovery a property of the system's design. Operator Undo intercepts and logs users' interactions with an e-mail service before they reach it, so an operator can rewind the service's state, repair it, and replay what users did since [6]. Operating system transactions let a program group system calls whose effects can be rolled back as a unit before they commit [35]. Intrusion recovery systems such as Taser log system activity so that an attack's effects can be identified and reverted afterward while legitimate changes are kept [11]. Each restores more completely than any list can, but each requires what this study's setting lacks: a service built to be undoable, a kernel and programs that use transactions, or continuous logging of all system activity.

A different route learns a command's effects by running it where they are contained. The *try* utility runs a command in a *semisolate* over the current environment, built from Linux namespaces and a union filesystem, captures its effects on the filesystem, and lets the user inspect them and apply, discard, or apply part of them [22]. Its authors motivate it with cases this study shares, among them a language-model-suggested `find -exec` that deletes a user's files. It is the closest prior work to the analysis's dry runs, but its knowledge comes from elsewhere. *try* executes the command itself, shows its effects to a user who must judge them, and does not contain effects outside the filesystem, such as network calls; the present study runs only commands it has first proven read-only, decides from the command's structure what it would destroy,

and captures exactly that beforehand. For the same reason, *try* is not scored as a baseline in Chapter 3: it decides nothing about a command, since it runs every command and leaves what to keep to the user.

The present study takes a narrower path between these literatures. Like a guard, it acts on one command before it runs, on an unmodified system and filesystem; like a recovery mechanism, it restores. It captures only what that command would destroy, as determined by analysis, so its cost follows what the command touches rather than the size of the system, and it can say beforehand whether undo will be possible. Table 2.1 compares the surveyed approaches on these properties.

Table 2.1. Approaches to the safety of shell commands, compared on the properties the study's algorithm combines.

Approach	**Acts on**	**Knows what will change**	**Composition and run-time targets**	**Restores what was lost**
Pattern-list guards: Codex CLI, cc-safety-net [21, 32]	The pending command	No: matches patterns	Partly: some wrappers and nested shells	No
Design proposal: NaSh [13]	The pending command	Proposed, mechanism unspecified	Not specified	Proposed, mechanism unspecified
Shell program analyses: ABash, Smoosh, CoLiS [2, 12, 28]	A script's text	Partly: CoLiS, for policy properties such as idempotence	Through their own semantics	No
Running in a semisolate: *try* [22]	The command, run over a union filesystem	After running it	Not needed	Yes: the user applies or discards
Agent workspace snapshots: ANOLISA [1]	An agent's workspace	No	Not needed	Yes: everything since the snapshot
Filesystem snapshots [37]	A whole subvolume	No	Not needed	Yes: all later changes
Versioning file systems [39]	Every file	No	Not needed	Yes: per file
Service undo, OS transactions, intrusion recovery [6, 11, 35]	A system built or logged for it	No	Not needed	Yes
This study	The pending command	Yes: its effect set	Yes: composed and resolved before it runs	Yes: exactly what the plan captured

2.6 Graphical and Computer-Using Agents

A parallel and larger body of work addresses computer-using agents[1] — systems that operate graphical environments by issuing actions and observing results. Although SynapseOS operates at the command line rather than by simulating graphical interaction, this literature is essential context: it establishes the dominant architectural patterns, defines the recurring technical bottleneck of visual grounding[2], and, through its platform coverage, reveals a gap in the setting where the analysis is deployed. The work is organized here by the platform each system targets — web, mobile, and desktop — together with the vision-language foundation models on which many depend.

Web agents

The web has been the most heavily studied environment for language-guided agents. Mind2Web, introduced by Deng and colleagues [10], is the first dataset for developing generalist web agents that follow language instructions across diverse real-world websites, comprising over two thousand open-ended tasks collected from 137 websites across 31 domains; its accompanying two-stage model uses a small language model to filter candidate elements from large documents before invoking a larger model to choose among them. WebArena, introduced by Zhou and colleagues [57], provides a fully self-hosted, reproducible web environment of functional website clones and evaluates 812 long-horizon tasks for functional correctness rather than trajectory similarity, establishing a challenging benchmark on which contemporary models achieved completion rates far below human performance. Zheng and colleagues [56], in their SeeAct framework, decouple visual understanding from action: the model observes a screenshot, generates a textual plan, and a separate step grounds that plan onto page elements. Their central finding is diagnostic for the entire field — with oracle grounding provided, the model completes 51.1% of tasks on live websites, substantially outperforming a text-only baseline at 13.3%, but performance degrades sharply when grounding must itself be inferred, confirming visual grounding as the principal bottleneck.

Mobile agents

Mobile-device control has produced large-scale datasets and progressively more autonomous agents. The pre-LLM work of Li and colleagues [23], discussed in §2.3, established the grounding problem for mobile interfaces. Android in the Wild, from Rawles and colleagues [36], contributes a dataset of 715,000 episodes spanning 30,000 unique instructions, in which actions are gestures at arbitrary screen coordinates rather than operations on indexed interface elements, exposing the challenge that actions must be inferred from visual appearance rather than from a structured model of the interface. AppAgent, from Zhang and colleagues [55], operates Android applications through a simplified action space of taps and swipes and learns to use new applications through autonomous exploration or by observing human demonstrations, generating application-specific documentation that matched or exceeded human-written documentation. Mobile-Agent, from Wang and colleagues [46], adopts a purely vision-centric approach that requires no system metadata, using optical-character-recognition and icon-detection modules to locate interface elements — achieving broad applicability across mobile environments at the cost of vision-grounding errors that require recovery mechanisms.

Desktop and cross-application agents

Desktop agents are the most direct graphical analogue to the present work, and their limitations are the most instructive. ScreenAgent, from Niu and colleagues [30], is a vision-language-model-driven agent for cross-application desktop control that operates in a plan–act–reflect cycle, specifying actions as coordinates and key sequences; its authors acknowledge that vision-only grounding is less reliable than approaches with API access. WinClick, from Hui and colleagues [17], targets Windows-desktop grounding and observes that most existing GUI agents rely on structured data formats such as the document object model or HyperText Markup Language, which are unavailable across general desktop environments; it therefore performs visual grounding directly from screenshots. UFO², the "Desktop AgentOS" of Zhang and colleagues [54], is the most ambitious desktop system surveyed: it fuses the native Windows accessibility framework

with vision-based parsing in a hybrid action mechanism, dispatching subtasks from a coordinating agent to application-specialized agents. Its authors argue that existing computer-using agents are hindered by shallow operating-system integration, fragile screenshot-based interaction, and disruptive execution; they identify adaptation to other desktop platforms, such as Linux and macOS, as future work, and frame cross-platform deployment as groundwork for a unified ecosystem of desktop automation solutions. UFO² acts on the user's behalf, as an automation layer beside the user's own Windows session, whereas the platform built for this study is operated by a person, turn by turn. AgentOS, from Liu and colleagues [25], goes further in ambition, proposing that a natural-language portal replace application silos as the way a person uses the operating system, but it is offered as a research direction, without a built system or an evaluation.

Contemporaneous commercial systems (as of October 2026)

The systems surveyed above are research artifacts. Shipping systems move faster than the peer-reviewed literature, and one is close enough to this study to warrant explicit positioning; what follows describes it as of October 2026. Alibaba Cloud Linux 4 Agentic Edition, ANOLISA, is a Linux distribution for AI agents whose default shell accepts ordinary language alongside shell commands and forces confirmation before operations it treats as unrecoverable, such as restarting the machine [1]; its checkpoint tool is described in §2.5. It is the nearest shipped system to the present work. It is built to run AI agents on cloud servers, whereas the platform studied here runs a local model under a person's direction, but the difference that matters here lies in recovery: it recovers by snapshot, restoring everything since a checkpoint, rather than by determining what one command will destroy and capturing exactly that. That difference reflects the question this study asks, which is not whether conversation is a better interface, but whether what a generated command destroys can be determined and captured before it runs; the platform's interface is the setting in which the answer is validated.

Vision-language foundation models

Underlying many of these agents are vision-language models trained specifically for interface understanding. CogAgent, from Hong and colleagues [16], is an eighteen-billion-parameter visual language model specialized for GUI understanding and navigation, using both low- and high-resolution image encoders to recognize the small icons and text typical of graphical interfaces. It supports the position that pure-vision GUI agents are viable when the underlying model is trained appropriately — but at a compute cost, and with the cross-platform generalization limitations common to the entire graphical-agent literature. Across web, mobile, and desktop, these systems demonstrate broad capability within a single platform class, while consistently identifying visual grounding as an unsolved bottleneck.

2.7 Accessibility and Speech-Driven Interaction

A distinct thread of the literature approaches conversational computing from the perspective of accessibility, targeting users with motor, speech, or cognitive needs. It is relevant to the present study in two ways: it provides the most rigorous human-centered evaluations in the field, and it exhibits a limitation that is the mirror image of the mainstream agent literature, and so it bears on the setting of the platform rather than on the analysis.

VoicePilot, from Padmanabha and colleagues [33], is a framework for using LLMs as speech interfaces to physically assistive robots for individuals with motor impairments. Although its immediate target is robotic assistance rather than desktop computing, it is included here because it is the most rigorous human-centered evaluation in the LLM-speech-interface literature. The authors derive five design guidelines from an iterative evaluation that ended in a study with 11 older adults at an independent living facility. Among them is that a command should execute in about the time a caregiver would take; the present study’s confirmation step adds to that time whenever it asks, which is one reason the study measures what asking costs as well as what it protects. DexAssist, from Mehendale and Walishetti [29], targets web navigation by users with

fine-motor impairments using two LLMs in tandem — one to interpret voice commands and a second to map the interpretation onto interface elements with fallback strategies — reducing the misinterpretation failures of single-model voice navigation on dense layouts. The architecture of van Dam [45] proposes retrofitting LLM-based conversational interfaces onto existing single-application graphical interfaces, exposing an application's navigation graph through a tool protocol and handling voice input through real-time speech recognition; its author observes that most production applications were never designed with speech in mind, making such retrofitting an ongoing challenge.

More recent work continues to mature the components required for speech-driven interface control. Han and colleagues, in GUIRoboTron-Speech [14], advance automated GUI agents driven directly by speech instructions, while Park and colleagues, in R-VLM [34], contribute region-aware visual grounding for more precise identification of on-screen elements. What unites this thread is that it operates at the level of the individual application or interaction technique rather than at the level of the session itself, presupposing an underlying conversational surface on which to run. A conversational interface layer over the whole session could provide one. Accessibility is accordingly positioned not as a target of the present study, which scopes it out for recruitment and ethics reasons, but as a possible later use of its platform.

2.8 Evaluation Environments and Benchmarks

The rapid proliferation of computer-using agents has been accompanied by increasingly rigorous evaluation environments, which establish both the current ceiling of the field's capability and the methods by which that capability is measured. The most consequential is OSWorld, from Xie and colleagues [50], a benchmark of 369 real-world computer tasks in a real-computer environment spanning Ubuntu, Windows, and macOS, and directly relevant to the present study as the source of a real-task suite against which benchmark comparability can be established. At its 2024 release the margin it measured was stark: humans achieved 72.36% success where the best evaluated model reached

12.24%. Its successor, OSWorld 2.0 [52], evaluates 108 long-horizon workflows whose median item requires roughly an hour and a half of skilled human operation, and there the best frontier configuration completes only 20.6% of tasks outright. The reported failures concentrate not in interface navigation but in tracking constraints across a long task, absorbing information that arrives mid-way, and verifying a result before declaring completion. The field's ceiling has moved substantially; it has not been reached. Direct comparisons between interfaces are rarer. AXIS, from Lu and colleagues [26], compares an API-first agent with unaided manual operation by people in Microsoft Word, in a within-subjects design with a cloud-hosted frontier model, and Zhou and colleagues compare screen-only graphical agents with command-line agents across 440 matched desktop tasks [58].

Complementing the benchmark literature, the survey of LLM-brained GUI agents by Zhang and colleagues [53] synthesizes the field through 2024, organizing it around the components of perception, planning, action, and memory, and informing the four-layer view adopted in §2.2 of this review. It enumerates the field's open challenges — accurate element localization, knowledge retrieval, long-horizon planning, and safety-aware execution control; of these, safety-aware execution control is the one closest to the present study. What the evaluation literature reveals, however, is a specific methodological limitation: rigorous as these benchmarks are, they measure the autonomous task completion of agents acting on a user's behalf, not the comparative performance of a human user working through a conversational interface versus a conventional graphical one. That distinction motivates the evaluation design reviewed in §2.10.

2.9 Synthesis: Comparative Analysis and Research Gap

The synthesis has two parts. The first, the gap the algorithm addresses, needs only a brief statement, because §2.5 has established it. Every shipped guard surveyed matches a command against a list of patterns; NaSh [13] proposes summaries of a command's effects without a mechanism; ToolEmu [38] measures risk across many runs

rather than guarding a command; every analysis of shell programs targets a property other than recoverability; and every recovery mechanism restores wholesale, as ANOLISA's snapshots do [1], or needs a system built for it, while *try* [22], not itself a recovery mechanism, previews a command only by running it and leaving its effects for the user to judge. None derives from a command's structure what it would destroy or captures what is needed to undo it: that is the research gap.

The second part concerns the setting in which the gap arises. Positioning the interface systems along the four-layer pipeline makes its structure explicit; Table 2.2 summarizes the principal systems by coverage scope, action mechanism, evaluation approach, and key limitation.

Table 2.2. Comparative analysis of surveyed systems on conversational and LLM-mediated interfaces for personal computing, positioned by coverage scope, action mechanism, evaluation, and key limitation.

Source	System	Coverage scope	Action mechanism	Evaluation	Key limitation
Wilensky et al. [49]	Unix Consultant	Shell consultation	Natural-language Q&A (no execution)	Demonstration, case studies	Hand-engineered knowledge base
Westenfelder et al. [47]	Westenfelder et al.	Shell (CLI)	Command translation	600-pair test set, fine-tuning	Limited system-context awareness
Gyawali et al. [13]	NaSh	Shell (Linux)	Guarded command translation	Worked examples, design paper	No formal user evaluation
van Dam [45]	Multimodal GUI Arch.	Single application	API binding + voice	Architectural prototype	Per-application retrofitting
Padmanabha et al. [33]	VoicePilot	Assistive robot	Voice + robot API	User study, 11 older adults	Robot domain, not the OS
Mehendale and Walishetti [29]	DexAssist	Web (e-commerce)	Dual-LLM voice + element mapping	Web-task evaluation	Narrow to e-commerce
Deng et al. [10]	Mind2Web	Web (137 sites)	Element selection from filtered HTML	Dataset benchmark	Structured-data-only in original form

Source	System	Coverage scope	Action mechanism	Evaluation	Key limitation
Zhou et al. [57]	WebArena	Web (self-hosted)	Functional task execution	Benchmark, low model baseline	Simulated, not live web
Zheng et al. [56]	SeeAct	Web (live sites)	Visual plan + grounding	51.1% with oracle grounding	Grounding hard without oracle
Rawles et al. [36]	Android in the Wild	Mobile (Android)	Coordinate gestures	Dataset, 715k episodes	Brittle to interface updates
Zhang et al. [55]	AppAgent	Mobile (10 apps)	Tap/swipe + element labels	50-task evaluation	Per-app knowledge acquisition
Wang et al. [46]	Mobile-Agent	Mobile (vision-only)	OCR + icon detection + planning	Mobile-Eval benchmark	Vision-grounding errors
Hong et al. [16]	CogAgent	Cross-platform GUI	Vision-language model (high-res)	SOTA on VQA / navigation	Compute cost of high-res inputs
Niu et al. [30]	ScreenAgent	Cross-app desktop	Vision + mouse and keyboard	Plan–act–reflect evaluation	Vision-only grounding fragility
Hui et al. [17]	WinClick	Desktop (Windows)	Visual grounding	WinSpot benchmark	Windows-specific
Zhang et al. [54]	UFO²	Desktop (Windows)	Hybrid API + vision	System deployment, task benchmarks	Windows-only; acts on the user's behalf
Xie et al. [50]	OSWorld	Ubuntu, Windows, macOS	Evaluation harness	369 tasks; 12.24% best vs 72.36% human at 2024 release	Reveals field-wide grounding gap
Li et al. [23]	PixelHelp / seq2act	Mobile (Pixel)	Two-stage transformer	ACL benchmark, three datasets	Pre-LLM; dataset-bound
Yao et al. [51]	ReAct	Cross-domain	Thought–action–observation prompting	Multi-benchmark evaluation	Inherits LLM hallucination, brittleness
Han et al. [14]	GUIRoboTron-Speech	GUI (speech-driven)	Speech-instructed agent	Benchmark evaluation	Application, not session, level
Park et al. [34]	R-VLM	GUI grounding	Region-aware visual grounding	Grounding benchmark	Component, not full system

Source	System	Coverage scope	Action mechanism	Evaluation	Key limitation
Jacobs et al. [19]	Local LLMs for NL2Bash	Shell (CLI)	Open-source model evaluation	Large-scale model comparison	Evaluation, not a deployed system
Lu et al. [26]	AXIS	Single application (Microsoft Word)	API-first agent, cloud-hosted frontier model	Within-subjects user study against manual operation	One application, one platform
Zhou et al. [58]	GUI vs. CLI agents	Desktop (440 matched tasks)	Screen-only GUI agents; CLI agents with and without verifier-guided skill augmentation	Agent benchmark	Agents, not people using an interface

Six patterns emerge from the comparison.

Bifurcation by coverage scope

The first pattern is a split by coverage scope. Shell-level work [13, 47] and single-application work [33, 45] sit at the narrow end; cross-application desktop, web, and mobile agents sit at the broad end, but each within one platform class. No system in the surveyed corpus implements a working conversational layer over the full graphical session that has been empirically validated against the conventional desktop workflow it would supplement. The Unix Consultant [49] came closest in ambition four decades ago, but its pre-LLM foundation kept it from scaling beyond demonstration, and the recent systems nearest that ambition do not fill the gap either: AgentOS [25] is a research direction without a built system, and UFO² [54] is an automation layer that acts on the user's behalf (§2.6).

Execution mechanism at the action layer

The second concerns the execution mechanism at the Action layer: how a system turns a decided intent into an effect, and what each choice costs. The literature divides among command translation, API binding, and GUI simulation, trending toward hybrid

execution [54] that prefers structured access where available and falls back to vision. Vision-only systems [30, 46] gain broad applicability at the cost of grounding fragility; structured-access systems gain precision at the cost of per-environment integration. The Unix Consultant, notably, worked purely at the consultation layer without executing actions, reflecting an underappreciated principle that the confirmation-gated design of the present study revisits: the agent that explains can be safer than the agent that acts.

Visual grounding as the principal bottleneck

The third is that visual grounding is the field's principal bottleneck, identified explicitly by both SeeAct [56] and the GUI-agents survey [53]: GPT-4V reaches 51.1% with oracle grounding, and its sharp degradation without the oracle is attributable in significant part to grounding failure. The bottleneck belongs to the graphical-simulation approach. A command-line approach, which grounds intent into shell commands rather than on-screen pixel regions, sidesteps it, at the cost of being bounded by what the command line can express. Direct comparison qualifies that tradeoff. Across 440 matched desktop tasks, screen-only graphical agents pass 59.1% while command-line agents reach only 48.2%, so the shell is not automatically the stronger substrate; but command-line agents whose skills are augmented under the guidance of verifiers reach 69.3%, the highest of the three conditions, and the authors attribute most of the command-line deficit to incomplete skill coverage rather than model capability [58]. The shell, then, is bounded by what its tools cover, which is the platform's concern; the analysis is concerned with what happens once a command is produced.

Scarcity of comparative user evaluation

The fourth concerns evaluation method: systems in this field are rarely tested with people, and seldom against the interfaces those people already use. Most surveyed systems are evaluated by architectural demonstration [13, 45] or curated task benchmarks [10, 46, 50, 57]; user studies are rare, and the comparisons that exist are narrow: VoicePilot [33] conducts a substantive human-centered evaluation of LLM-based

interaction, and AXIS [26] compares an API-first agent against unaided manual operation in a within-subjects design, but within a single application, on a single platform, with a cloud-hosted frontier model. No surveyed work compares a conversational interface against the conventional graphical workflow across operating-system-level tasks on the same population. That gap belongs to the setting of this study, and the validation study addresses it in passing rather than as a research question.

The operating-system substrate

The fifth concerns the operating-system substrate: the Linux desktop, as the person using the machine meets it, is a platform the surveyed work leaves unserved. Windows [17, 54], Android [36, 46, 55], and the web [10, 56, 57] are each heavily represented, while the Linux work that exists is aimed elsewhere than the end user's desktop session: NaSh [13] addresses Linux at the shell level as a design proposal; Alibaba Cloud's ANOLISA ships a natural-language Linux terminal with approval gating and rollback, but scoped to server and DevOps workloads; and OSWorld [50] includes Ubuntu as an evaluation platform rather than a deployment target. No surveyed system implements a conversational session layer for the Linux desktop addressed to the person using the machine, despite Linux's wide use for development and server work.

Shared lineage and its fragility

The sixth is the field's shared intellectual lineage and its fragility. As §2.3 established, nearly every post-2023 system inherits the ReAct [51] convention of interleaving reasoning with action, which explains both the field's rapid progress and the pervasiveness of LLM-reasoning failure modes across it. That fragility makes safety and recoverability mechanisms, rather than raw capability alone, decisive for any deployable system.

One related but distinct class needs explicit positioning: AI-assisted coding agents, which translate natural-language intent into shell commands and execute them under human oversight, the same basic mechanism this study adopts. They are excluded

from Table 2.2 not because they are industry rather than academic work (a substantial share of the literature surveyed here is likewise unrefereed preprint) but because they differ on every axis that matters here: they are built for software engineers working within an existing code repository and powered by large models on remote infrastructure, whereas SynapseOS is addressed to any user of the machine regardless of command-line fluency and runs a locally hosted small language model. The mechanism is shared; the population and the model's location are not.

These patterns converge on a fourfold research gap. *First*, the shipped systems surveyed guard a generated command by permission, matching it against an enumerated list of dangerous patterns, while recovery restores wholesale, needs a system built for it, or, in *try* [22], runs the command for the user to judge; none determines before a command runs what it would destroy and captures exactly that (§2.5). This is the gap the study's algorithmic contribution addresses. *Second*, no system in the surveyed corpus provides a conversational interface as a general route to operating a personal computer: addressed to the person using the machine, spanning the operating system rather than one application, and evaluated as an interface rather than as an autonomous agent. Existing systems are automation utilities that act on the user's behalf, measured by how often they complete a task unattended, not by whether a person directs them more effectively than the interfaces they already have. *Third*, the work that exists on an open platform targets servers, agents, and benchmarks rather than an end user's desktop session. Linux is chosen because its desktop session, toolchain, and command line are open to inspection and modification, making it the condition of the experiment rather than its subject; the missing end-user desktop work there is what the platform addresses. *Fourth*, controlled comparison of a language-model-mediated interface with the conventional graphical workflow, with people, exists only within a single application, on a single platform, with a cloud-hosted frontier model [26]; no such comparison was found in the surveyed corpus for operating-system-level tasks under a locally-hosted model and an explicit safety gate.

SynapseOS is positioned against the other three gaps, as the platform on which the analysis addressing the first is validated. It is a conversational interface layer for the Linux desktop that translates natural-language intent into shell commands under a confirmation gate driven by the analysis, and it is evaluated against each participant's own operating system. The closest prior work, NaSh [13], proposes guardrails for an LLM-powered natural-language shell (generated code that remains analyzable, selective undo, and summaries of what a command would affect) as a design position, explicitly leaving their mechanisms and evaluation as open work. The present study contributes a mechanism for part of that position: an analysis that derives what a command would affect from its structure, through composition and the read-only resolution of run-time targets, and a plan that captures the least that undoing it requires, evaluated against ground truth observed wherever a command acts on files.

2.10 Review of the Methodological Literature

The methodological literature bears on the study's two kinds of evidence in turn. The primary one, the evaluation of the algorithm, follows established practice for judging program analyses. A sound analysis is judged first by what it misses, its false negatives, and only then by what its false alarms cost, since soundness is the property it claims [8]; silent loss is therefore the primary measure and needless confirmation the cost. Ground truth for every command that acts on files is observed by execution, not assigned by judges, and the 80 commands whose effects lie outside the filesystem are labeled from a fixed table. Execution is the standard by which executable semantics of the shell are validated against what shells actually do [12], and it underlies the scoring of generated Bash commands against what was intended [47]. Part of the corpus comes from a public dataset, NL2Bash [24], so the evaluation does not rest only on commands written by the analysis's own authors. Ablation, rerunning the analysis with each component switched off, tests the claim that particular components produce the result. Comparisons scoring every system on the same commands use the standard paired-proportion tests: McNemar's exact test on the commands two systems handle differently,

Wilson intervals for each proportion, Newcombe intervals for each difference, and Holm's correction across each family of tests. Chapter 3, Sections 1.2, 3.1, and 3.2 give the design.

The second kind of evidence comes from the validation study, in which people use the analysis deployed in an interface, so its evidence turns on how a conversational interface should be evaluated. The surveyed work evaluates in three ways, each with a characteristic limitation. Architectural demonstrations [13, 45] establish feasibility but give no performance evidence. Curated task benchmarks [10, 50, 57] give reproducible, quantitative measures, but of agents completing tasks autonomously on a user's behalf, not of a human working through an interface. Human-centered user studies, exemplified by VoicePilot [33] and AXIS [26], give the most direct evidence of an interface's value to real users, but they are rare in this literature and, where present, confined to a single application rather than the operating system as a whole.

These limitations motivate a design grounded in established human-computer-interaction practice. It is *within-subjects*: each participant completes tasks under both SynapseOS and their own operating system, serving as their own control so that between-person variance is removed as a confound, with the order of conditions counterbalanced to distribute learning effects evenly. Its *expert baseline* is each participant's most familiar environment rather than a fixed, possibly unfamiliar alternative, so no participant is measured on an environment they have not used. This strengthens the ecological validity that the surveyed benchmark-only evaluations lack; because familiarity then favors the baseline, however, the comparison characterizes the platform against each participant's practiced environment rather than isolating interface quality (Chapter 3, Section 1.2(d)). The instruments are established rather than constructed for the study: the System Usability Scale [5] for perceived usability, interpreted through subsequent large-sample analysis [40], and the NASA Task Load Index [15] for perceived workload. A *convergent mixed-methods* structure in the sense of Creswell and Plano Clark [9] collects the quantitative measures of task completion time and error rate and the qualitative evidence of user experience concurrently and integrates them at interpretation. The

qualitative data, from retrospective think-aloud over the screen recording and a semi-structured interview, are analyzed with Braun and Clarke's thematic analysis [3], a widely adopted six-phase approach for identifying and reporting patterns in qualitative data. Finally, the reviewed systems shape what is measured: the reversibility-and-guardrail framing of NaSh [13] and VoicePilot's guideline that a command should take no longer than a caregiver would [33] together motivate measuring both what the confirmation gate and undo protect and what they cost the participant.

Chapter 3 details how this methodology is instantiated: the benchmark and custom task suite, the participant profile and screening criteria, the session procedure, the metrics and their computation, and the statistical and qualitative analysis plan.

2.11 Summary

This chapter reviewed two literatures. The first, on which the study's algorithm builds, is program analysis, with abstract interpretation and type-and-effect systems [8, 20, 44] as its formal basis. Around it, the command guards of natural-language shells and coding agents [13, 21, 32] decide by permission rather than by effect; the static analyses of shell programs [2, 12, 28] target properties adjacent to recoverability but not recoverability itself; recovery mechanisms, from snapshots, including ANOLISA's [1], to operating system transactions [6, 11, 35, 37, 39], restore after the fact, but wholesale or only on a system built for it; the *try* utility [22] previews a command by running it; and ToolEmu [38] measures the risks of agents rather than guarding a given command. The second literature, in which the platform is situated, covers nearly four decades of natural-language and LLM-mediated interfaces to personal computing, organized along a four-layer pipeline of Input, Reasoning, Action, and Operating-System Integration. Its foundations set the long-standing aspiration to a natural-language interface to a real operating system [49] and the methodological shift, driven by in-context reasoning [51], that made the current generation of systems possible. The critique of natural-language interaction from direct manipulation [7, 18, 27, 31, 41–43], with the controlled comparison of Whiteside and colleagues [48], cautions that a conversational interface

must earn its place, which shapes the platform's design. The shell thread, most directly related to the proposed design, showed that natural-language-to-Bash translation is tractable [47], that safety and reversibility are central [13], and that local deployment is feasible [19], leaving the extension from shell to session, and empirical user evaluation, open. The graphical-agent literature across web, mobile, and desktop [10, 16, 17, 30, 36, 46, 54–57] set the field's architectural patterns and its recurring bottleneck of visual grounding, and the accessibility thread [14, 29, 33, 34, 45] revealed a complementary limitation a general conversational interface layer could address. Finally, the evaluation literature [50, 52, 53] traced a capability ceiling that remains low on long-horizon work and a methodological gap: interfaces are seldom evaluated comparatively against the conventional workflow with real users, and, in the surveyed corpus, not at the level of the operating system itself.

Taken together, the surveyed works reveal the fourfold gap set out in the synthesis: no analysis of what a generated command would destroy, with every shipped guard deciding by permission, every shipped recovery restoring by snapshot, and *try* [22] leaving the judgment of a contained run to the user; no conversational interface over a whole desktop session for the person using it; no coverage of the Linux desktop; and no controlled comparison of a conversational interface against the conventional workflow at the level of the operating system. The study's research addresses the first through a recoverability analysis of generated shell commands, evaluated against ground truth from sandboxed execution wherever a command acts on files; the other three describe the setting in which that gap arises. SynapseOS, the conversational interface layer for the Linux desktop built as the analysis's experimental platform, is where the analysis is deployed and validated in use, in a controlled, within-subjects comparison against each participant's own operating system. Chapter 3 sets out that methodology.

Notes

1. A computer-using agent is an artificial-intelligence system that operates a computing environment by issuing actions and observing their results. A GUI agent is a computer-using agent whose action space consists of operations on a graphical user interface — typically clicks, key presses, scrolls, and text entry — usually driven by a large language model or a multimodal (vision-language) model that additionally accepts images as input.

2. Grounding is the problem of mapping a model's textual plan onto a concrete operation in the environment. Visual grounding is the specific subproblem of identifying the on-screen pixel region corresponding to a referenced interface element — repeatedly identified in the surveyed literature as the principal bottleneck for graphical agents, and one that a command-line approach avoids by grounding intent into shell commands rather than into screen coordinates.

Chapter 3

METHODOLOGY

This chapter describes the methodology for designing, implementing, and evaluating the recoverability analysis, and for deploying and validating it in SynapseOS, the conversational interface layer that serves as its experimental platform and runs over an ordinary Linux desktop session rather than replacing it. The methodology is organized into three sections: Dataset, which defines the data to be collected and how it will be gathered; Procedure, which describes how the analysis and its platform are built and how the validation study is conducted; and System Testing, which defines the evaluation of the analysis on its corpus and the design, metrics, and statistical approach of the validation study.

This methodology answers the five research questions posed in Chapter 1, all of which concern the analysis. *RQ1* asks how a command's effect on the system can be represented formally so that recoverability is a predicate over it, with anything undetermined treated as unrecoverable; *RQ2*, whether those effects can be derived from the command's structure before it runs, through composition and the read-only resolution of run-time targets; *RQ3*, whether a minimum-cost recovery plan computed from them restores the prior state; *RQ4*, whether the analysis loses less data silently, restores more, and asks less often than a pattern list, a fail-closed list, and the command-safety checks of open-source AI coding agents; and *RQ5*, how much composition and runtime-target resolution each contribute beyond the per-command rule table. RQ1 is answered by the effect model specified in Section 2.1(a), with the algorithm and the plan it supports specified in (b) and (c); RQ2 through RQ4 are answered by the evaluation against baselines in Section 3.1, and RQ5 by the ablations in Section 3.2.

Two evaluations follow, of different standing. The corpus evaluation in Sections 3.1 and 3.2 is the study's primary evidence: it answers the research questions against a labeled corpus whose ground truth comes from sandboxed execution, without reference to any participant. The validation study, defined in Section 1 and Sections 3.3 onward and run on the platform described in Section 2, deploys the analysis in SynapseOS and has participants use it on matched tasks. It checks that the analysis's decisions carry through to real files and hold on commands people cause, which the corpus does not contain, and it characterizes the platform through usability measures. Neither evaluation depends on the other: the analysis takes a shell command as a string and is indifferent to what produced it, so removing the interface would remove the validation study but leave every research question, its corpus, its baselines, and its results intact.

Conceptual Framework

The conceptual framework places the recoverability analysis between a generated shell command and its execution (Figure 3.1). SynapseOS supplies the setting: the user issues a request in ordinary language through a persistent chat interface, a locally hosted small language model translates it into a shell command, the command is executed in a bash subprocess, and its output is rendered back in the interface. Before execution, the analysis takes as input the command and the directory in which it will run. It derives the command's effect set from its structure (Objectives 1 and 2), learning targets known only at run time through dry runs it first proves read-only and runs in a read-only sandbox; it decides whether the command can be undone; and, where undoing it requires preparation, it computes the minimum-cost set of pre-images to capture (Objective 3). Anything it cannot determine makes it ask. The analysis's verdict, rather than a pattern match, drives the confirmation gate; its plan's captures are taken before the command runs and recorded in an undo journal; and undo restores from them. Because the confirmation gate requires the user to approve irreversible operations, the generated shell command is displayed rather than hidden: an approval given without sight of what is

being approved is not consent. Visibility also makes the system's translation of intent observable to the participant, which is what allows a user to notice a misinterpretation before it executes rather than after. The analyzer is specified in Section 2.1(a)–(c) of this chapter and evaluated in Sections 3.1 and 3.2, independently of the validation study.

The framework is evaluated in two parts of different standing. The recoverability analyzer, the study's research contribution, is evaluated on its own, on a corpus of commands whose ground truth is obtained by sandboxed execution (Sections 3.1 and 3.2). It is then validated in use, deployed in the platform, through a controlled study in which participants complete a fixed set of tasks under two conditions: using SynapseOS as the interface, and using their own primary operating system — Windows, macOS, or Linux — as the baseline. The study records the analyzer's decision on every command participants cause, so that its behavior in use can be compared with its behavior on the corpus. Its expert baseline design, which pits SynapseOS against the environment each participant already knows and uses daily, also characterizes the platform through task completion time, error rate, and user satisfaction across the real-world OS landscape [50].

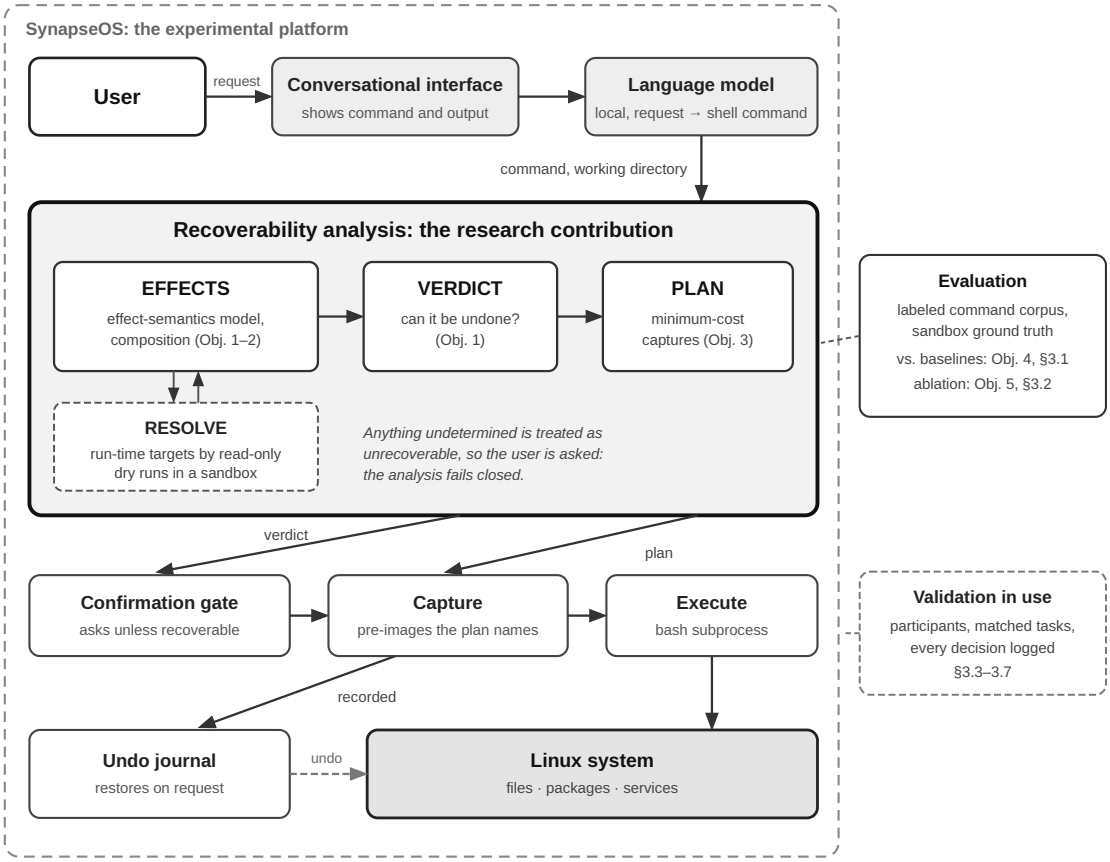


Figure 3.1. Conceptual framework: the recoverability analysis and the platform that deploys it. A request in ordinary language is translated by a local language model into a shell command. Before the command runs, the analysis derives its effect set (EFFECTS, Objectives 1–2), learning targets known only at run time through read-only dry runs in a sandbox (RESOLVE); decides whether it can be undone (VERDICT); and computes the minimum-cost set of pre-images to capture (PLAN, Objective 3). Anything it cannot determine makes it ask. The verdict drives the confirmation gate; the plan's captures are taken before execution and recorded in the undo journal, from which undo restores. The analysis is evaluated on a labeled command corpus (Sections 3.1–3.2) and validated in use with participants (Sections 3.3–3.7).

System Environment Specifications

This section separates three things that are easily confused: the minimum hardware SynapseOS is built to run on, how its language model is sized to the machine, and the larger machines used to develop and test it. It then fixes the machines on which

the study itself runs. The user study is conducted across three dedicated machines — one running SynapseOS (Condition A) and two covering the three supported baseline operating systems (Condition B). Table 3.1 describes each machine's role and operating environment. Exact processor, RAM, and storage specifications for each machine are confirmed and documented in the study protocol prior to the pilot study. Table 3.2 specifies the minimum hardware and software requirements for deploying SynapseOS independently of the study configuration.

Table 3.1. Study evaluation machines

Machine	Role	OS	Notes
SynapseOS evaluation machine	Condition A — all participants	SynapseOS	GUI mode; fullscreen conversational interface running within an XFCE (X11 session) desktop environment; based on Debian 13 (Trixie); Ollama running locally the language model for the machine's memory tier (Table 3.2), the tier being fixed before the pilot study from the model comparison of Section 2.1(d); a participant-accessible fallback returns to the underlying XFCE session if needed, logged and excluded from the primary analysis (Section 3.7)
Windows / Linux dual-boot baseline machine	Condition B — Windows- or Linux-primary participants	Windows 11 or Ubuntu 24.04 LTS (GNOME), selected at boot	Standard consumer x86-64 hardware; Windows 11 and Ubuntu installed on separate partitions; standard user account per OS; no custom software. Facilitator boots to the OS matching the participant's assignment before the session begins.
macOS baseline machine	Condition B — macOS-primary participants	macOS (current release)	Apple Silicon hardware; standard user account

Table 3.2. SynapseOS minimum deployment requirements. SynapseOS is distributed as a Linux system based on Debian 13 (Trixie).

Component	Minimum Requirement

Processor	64-bit x86 processor, 2-core minimum; no GPU required
RAM	4 GB (CLI mode[1] / TUI mode[2]); 8 GB minimum (GUI mode)
Storage	10 GB available at the minimum tier (operating system, language model, and application); more on larger tiers, depending on the model chosen (Section 2.1(d))
Display	CLI mode / TUI mode: any terminal or remote connection; no graphical environment required. GUI mode: graphical desktop environment required.
Language Model	Locally hosted code language model sized to the machine's memory, leaving the desktop, a web browser, and everyday applications the memory they need; CPU-only inference; no GPU required. At the minimum tiers, 4 GB for the terminal modes and 8 GB for GUI mode, this is a small model (~1.8 GB)[3]; machines with more memory may run a larger one (Section 2.1(d)). A cloud language model backend is planned as an optional, user-configured alternative; it is not implemented in the thesis prototype.
Shell	GNU bash. Commands generated by the intent parser are dispatched through `bash -c`; a POSIX-only shell such as dash is insufficient, since generated commands may use bash-specific syntax.
Filesystem	ext4 root, the Debian default. It has no cheap snapshots or reflinks, so a full copy of a working directory before each command costs time and space proportional to its size; the capture plan exists to avoid that cost. A copy-on-write filesystem such as btrfs or XFS would make capture cheaper and is left as future work.
Sandbox	bubblewrap. The analysis runs its read-only resolvers with the whole filesystem mounted read-only inside it. Without it, resolution is switched off and any command that needs it asks, so the system stays safe and only becomes more cautious.
Inference Server	Local inference server (Ollama)[4]; manages model lifecycle, loading, and query serving on the host machine.

The minimum is a design commitment, not the environment the work is done in. SynapseOS is built to run on the smallest machines a person is likely to own, but it is developed and tested on larger ones: a server with enough memory to run every model tier side by side, each in its own virtual machine sized as Table 3.2 describes and loaded with a web browser and everyday applications, so that tiers are compared under the load

a person would actually put on them; and disposable, snapshotted machines on which destructive commands can be run for real and the system restored. Nothing about this environment is a requirement for running SynapseOS.

SynapseOS offers three interface modes, all of them deliverables of this study. The user study evaluates SynapseOS in GUI mode, which requires a graphical desktop environment and at least 8 GB RAM. TUI mode is a terminal-based interface requiring no display server, running on as little as 2 vCPU and 4 GB RAM, and is designed for server and remote deployment environments accessible over SSH; CLI mode[1] serves scripting and automation and passes each command through the same confirmation gate (Section 2.1(e)). CLI mode is not evaluated in the user study, and TUI mode only under the fallback described next. The study condition does, however, depend on GUI mode being complete and stable in advance of participant recruitment, and a slipped implementation milestone would otherwise delay the entire evaluation. The condition is therefore fixed at a defined readiness checkpoint prior to ethics submission: if GUI mode is not feature-complete and stable at that checkpoint, the study is conducted in TUI mode instead, which is functionally equivalent in every respect the research questions address — the same conversational surface, the same execution pipeline, the same confirmation gate and undo journal, differing only in the rendering layer and in requiring no display server. The mode actually used is reported with the results. Throughout Condition A, the underlying XFCE session continues running beneath the conversational interface, and if SynapseOS becomes unresponsive during a task or a participant wishes to stop using it, a fallback returns them to that session. Every fallback invocation is logged, and any task during which it occurs is excluded from the primary completion-time and error-rate comparison rather than counted as a SynapseOS success (Section 3.7).

Section 1: Dataset

The study gathers two kinds of data. The primary dataset is a held-out corpus of shell commands, each labeled by running it in a sandbox, on which the analysis is evaluated against its baselines and its ablations (Objectives 4 and 5). The remaining data

serve the validation study, in which the analysis is deployed in SynapseOS and exercised by participants, in three categories: benchmark task data used to exercise the platform in a standardized environment, user study data collected from human participants in a controlled setting, and system performance data, including the analysis's decision on every command, captured automatically through built-in telemetry during all sessions.

1.1 What to gather

a) Command Corpus

The primary dataset is a corpus of shell commands, held as four partitions: human-written commands from NL2Bash [24], templates covering each shell composition operator, commands whose effect lies outside the filesystem, and commands the deployed model itself wrote. Each command is paired with a fixture — the files and directories that exist when it runs — and with a ground-truth label observed by running it on that fixture in a sandbox: whether it loses nothing, loses something a captured pre-image could restore, or acts outside the filesystem. Every configuration compared in Section 3 — the analysis, its baselines, and its ablations — records its decision on each command: whether it would ask, and what it would capture. Every command the analysis would let run is also executed and undone and the tree compared, which records whether the capture restored the prior state.

b) Benchmark Task Suite

Tasks come from two suites: an established public benchmark, which ties the platform's results to the literature, and a custom suite written for this study. The OSWorld task suite [50] provides standardized real-computer tasks across shell operations, file management, web browsing, and application-level interactions. It evaluates computer-using agents in real operating system environments: tasks are presented as natural language instructions, agents interact with a real virtual machine

environment, and outcomes are evaluated by automated inspection of the resulting system state — whether the goal was actually achieved — rather than by comparing individual actions. Here it serves as a secondary check that relates the platform's results to the existing literature, and only its tasks that can be expressed as shell commands are adapted, since graphical-only tasks are outside SynapseOS's scope (Section 2.1(e)). Alongside OSWorld, a custom cross-platform task suite is developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing; each task is defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine. Each task in both suites is documented with a description, expected outcome, task category, and difficulty level.

c) User Study Data

Each participant yields two kinds of evidence: how they performed, and how the experience felt to them. The following data types are collected from each participant during the user study:

- **Task completion time:** the elapsed time from when the task prompt is presented to when the participant signals task completion or the session logs a terminal event
- **Error rate:** the proportion of tasks presented that end in failure or an incorrect outcome, per participant per condition; recovery attempts are recorded separately as an error category rather than counted in the rate
- **User satisfaction scores:** responses to the System Usability Scale (SUS) [5] and the NASA Task Load Index (NASA-TLX) [15], collected immediately after each condition; both are validated instruments for measuring perceived usability and cognitive workload respectively
- **Qualitative feedback:** responses gathered through a semi-structured post-study interview covering perceived ease of use, trust in the system, and comparison between the two conditions

d) System Performance Data

The platform also records its own behavior, including every decision the analysis makes. The following telemetry data is captured automatically by SynapseOS throughout all sessions:

- Command execution latency: time elapsed from utterance submission to bash subprocess completion
- Undo log events: count and type of operations logged to the undo stack
- Confirmation prompts triggered: count of irreversible operations that required user confirmation
- Analysis decisions: for every command executed, the analysis's verdict and capture plan, whether the participant was asked, the task fixture it ran against, and a snapshot of the task directory taken immediately before the command ran, so that the command can be replayed on the state it actually met and scored as the corpus is (Section 3.5(e))
- Conversational turns per task: the number of utterances required to complete each task
- Intent parsing accuracy: whether the system correctly identified the user's intent on the first attempt

1.2 How to gather

a) Command Corpus

The corpus is generated by a program, not assembled by hand, and its four partitions are reported separately rather than pooled. The first, NL2Bash [24], supplies human-written commands and serves as the generalization set; candidates are drawn from it at random with a recorded seed, excluding any command used during development. The second is a set of parameterized command templates covering each shell composition operator — sequencing, conditional execution, pipelines, redirects, command substitution, loops, `find -exec`, `xargs`, `sh -c` and other interpreters, and

wrappers such as `sudo` and `env` — because naturally occurring corpora under-represent compound commands, and compound commands are where the algorithm's contribution lies. The third holds commands whose effect lies outside the filesystem, described under (b). The fourth holds commands the deployed model itself wrote: the prototype's model is given the NL2Bash descriptions of a random sample of tasks with the runtime's own prompt, and the command it returns is labeled by the same pipeline. This partition is reported apart for two reasons: drawn from this prototype's own output, it is biased toward the shapes the prototype produces; and the analysis is about model-generated commands, which NL2Bash's human-written ones are not. Commands logged during the validation study are scored the same way (Section 3.5(e)).

The corpus is also stratified by shape, which the generator assigns from each command's syntax: *single* (one named command), *chain*, *run-time* (targets computed while the command runs), *redirect*, *hidden* (the effect lies inside a shell, interpreter, or wrapper the command names, such as *sh -c*, *awk*, or *env*), and *external* (the effect lies outside the filesystem). A command that fits several shapes takes the first that applies, in the order hidden, run-time, chain, redirect, single. Because shape follows syntax alone, a program the rule table does not model is classed by how it is written, not by why the analysis asks about it; this is why the 34 losses the analysis asked about are counted differently by cause and by shape in Section 3.1(d). Results are reported per shape as well as in aggregate, because the claims under test concern the composed, run-time, and hidden shapes, and a corpus dominated by plain named commands would measure the strength of a list, not the contribution of the analysis.

b) Ground Truth by Sandboxed Execution

Ground truth is observed, not judged. Each executable command is given a fixture — the files and directories that exist when it runs — and is run once in a bubblewrap sandbox where only the fixture is writable, the rest of the filesystem is read-only, there is no network, standard input is empty, and a hard time limit applies. The tree is recorded before and after, and the label follows from the difference. A command is

recoverable when nothing that existed was lost: paths may have been created, and a path that disappeared while its very same file now sits elsewhere was moved, not lost. It is *recoverable only if a pre-image is captured* when something that existed was removed, overwritten, replaced by a different kind of object, retargeted, or given different permission bits or an owner. Modification times are ignored, since touching a file loses nothing a user could want back. Commands that do not finish cleanly (a non-zero exit, a timeout, an attempt to write outside the fixture) are excluded and counted, never labeled by guess.

Commands whose effect the filesystem cannot show — on another machine, the mount table, another process, a device, an account, or the firewall — cannot be labeled this way and are never executed; they form the external partition and are labeled *unrecoverable* by construction. The rule that assigns this partition is deliberately narrow, naming only tools that change such state, so that a read-only network query is not counted as a loss the system failed to catch. It assigns the partition and is not a source of the other two labels.

Ground truth is defined relative to a stated filesystem state, because a command's effects depend on what exists: *rm *.tmp* deletes nothing where there are no such files, and *cp a b* overwrites only if *b* is there. A label given with no state describes an imagined directory, against which an analysis that reads the real state cannot be scored. Every corpus item therefore carries its fixture, and the analysis runs against that same fixture. The fixture is built from the command's text before the analysis is consulted: the files and directories the command names are created, a glob becomes a few matching files, and whether the destination of a copy, move, or redirect already exists is chosen by the seed, with the other choice tried if the first does not run. Fixtures are never written from the analysis's output.

The generator shares no code with the analysis and never consults it, so agreement between the two is evidence, not circularity. It is deterministic for a given seed and records beside the corpus the number of candidates drawn and the reason for

each exclusion. With no human labeler there is no labeler disagreement to measure; the threat that takes its place, the validity of the oracle itself, is addressed in Section 3.1(e).

c) Benchmark Tasks

The benchmark tasks are run through SynapseOS as a user would give them, with two adjustments: they are adapted to the platform's Debian system, and a facilitator attends every run. SynapseOS is deployed on Debian 13 (Trixie), whereas OSWorld's own environment is an Ubuntu virtual machine [50], so OSWorld's shell-expressible tasks are adapted to the Debian evaluation machine. Each task is given to SynapseOS as a natural language instruction through the conversational interface; results are captured by the built-in telemetry, and the resulting system state is compared with the outcome the benchmark defines. Because SynapseOS asks before every command its confirmation policy does not let through unasked (Section 2.1(e)), benchmark runs are facilitator-attended, not unattended, with the confirmation gate active throughout and every confirmation logged in the run record. OSWorld scores the resulting system state, not a human operator's performance, so it applies only to the SynapseOS condition; Condition B, which a participant operates directly, has no equivalent and is evaluated instead through the custom cross-platform task suite of the user study (Section 1.2(d)).

d) User Study

The user study measures each participant twice, once on SynapseOS and once on their own operating system, so that the main comparison is made within the same person. The user study takes place in a controlled laboratory with two machines: the SynapseOS evaluation machine (Debian 13, conversational interface) and a baseline machine running the participant's native OS — Windows 11, macOS, or Linux with a GNOME desktop — as determined at screening. Under a within-subjects design each participant completes a matched task set on both machines, and because repeating an identical task would measure recall rather than interface efficiency, there are two parallel task forms, Set A and Set B, matched item for item on category, difficulty level, and

number of required steps and differing only in the specific targets involved. Form and condition order are crossed in a 2×2 counterbalance, giving four groups of ten, one for each combination of the form done first (Set A or Set B) and the machine used first (SynapseOS or the native OS). Participants are allocated to groups by block randomization from a recorded seed, in blocks of four stratified by primary OS, so that order and form are not confounded with proficiency or operating system; task order within a form is randomized per participant. Form equivalence is verified in the pilot study, and condition order is retained as a between-subjects factor in the analysis. Counterbalancing redistributes carryover but does not remove it, and asymmetric transfer — carryover larger in one order than the other — cannot be separated from the order effect in a two-period design; it is therefore limited by the parallel forms and stated as a limitation rather than tested (Section 3.7). Screen capture records both machines, supplemented in the SynapseOS condition by its session logger.

The study involves 40 participants spanning the range of computing proficiency: from **novice users**, recruited from the general university population, who have limited computing experience, can perform only basic tasks such as web browsing and document editing, and have no command-line familiarity; through everyone in between; to **power users**, recruited from the university's information technology and computer science programs and proficient on their primary OS in file management, system monitoring, and process administration. No participant is assigned to a group: each is measured by the proficiency score of Section 1.4, and every enrolled participant is analyzed.

Participants are recruited through university channels, only after ethics approval is secured. Recruitment enforces a minimum of four of the forty per primary OS (Windows, macOS, Linux), so that the sample cannot concentrate in one OS and leave the per-OS exploratory subgroup analysis (Section 3.4) without data for the others. It also aims for at least fifteen of the forty reporting basic and at least fifteen reporting advanced proficiency on their primary OS, so that the proficiency score of Section 1.4 spans its range. Within these quotas, eligible volunteers are enrolled first-come until forty are reached, and a quota still unmet at the end is reported as a limitation. Because a

daily Linux user with basic proficiency is uncommon, the Linux quota is met from more proficient users if it cannot include one, and the resulting imbalance is reported rather than concealed. Consent has two stages: a short screening consent at sign-up, covering the screening questionnaire and screener call, how the volunteer's data are kept, and their destruction if the volunteer is not enrolled; and the full informed consent (Appendix F) at the start of the session. Participant data are pseudonymized at collection under the participant ID scheme of Section 2.5; recordings, which cannot be anonymized at collection, are governed by the retention and destruction rule stated there.

Participants complete the SUS and NASA-TLX questionnaires after each condition, and a brief semi-structured interview after both gathers qualitative feedback.

1.3 Why

The analysis is evaluated against a labeled corpus of shell commands rather than through participants, because its research questions concern what a command does, which can be observed directly, and not what a person does. The separation is deliberate, and it has a practical consequence: this evaluation depends on no ethics approval, no recruitment, and no participant sessions, so it can be completed and reported independently of the validation study's schedule.

The corpus is sized against the classes the analysis has to separate, not against a single overall total. Commands are drawn until a fixed number carry each ground-truth label, so that the label the false-negative test needs — a command that loses something — is present in quantity whatever NL2Bash's own composition, which is mostly read-only. The target is on the order of 780 commands: about 450 from NL2Bash, of which 150 are losses; about 250 from templates; and about 80 external. This is a coverage-based target rather than a power-computed one: a classical power analysis needs an assumed effect size, and the quantity of interest, the false-negative rate, is expected to be low for both the baseline and the algorithm, the regime in which an assumed effect size would be a guess. What the corpus must give is enough losses that a Wilson interval on a proportion near zero remains informative, and 150 does.

The OSWorld benchmark serves as a secondary check on the platform, for the reason given in (b) above: it relates the platform's results to the existing literature. The analysis itself is evaluated on the command corpus, for the reasons just given.

A custom cross-platform task suite is necessary because OSWorld's task coverage does not represent the human-goal-framed system tasks relevant to this study — file organization, system monitoring, process management, and text processing — achievable both through SynapseOS natural language commands and through participants' native OS tools (Windows, macOS, or Linux graphical desktop). Tasks are scoped to operations achievable via bash on SynapseOS and via native graphical or terminal tools on the comparison machine.

The evaluation follows a convergent (parallel) mixed-methods design: the quantitative performance measures — task completion time, error rate, SUS, and NASA-TLX — and the qualitative interview data are collected concurrently within each session and integrated at the interpretation stage, so that the measured differences between conditions and participants' accounts of why those differences arise inform one another rather than being read in isolation.

The within-subjects design is chosen because it allows each participant to serve as their own control, producing a direct comparison between conditions while requiring a smaller participant pool than a between-subjects design. Counterbalancing addresses the primary threat to internal validity, the learning effect, by distributing it across conditions; it does not remove asymmetric transfer, which the parallel forms limit and Section 3.7 states as a limitation.

The expert baseline design — using each participant's primary OS as Condition B rather than a fixed Linux graphical desktop — is chosen because a fixed Linux baseline would confound interface quality with OS familiarity. The overwhelming majority of participants have never used a Linux desktop, so performance under that condition would reflect unfamiliarity with a foreign environment, not a meaningful comparison of interface paradigms. Measuring the platform against what participants already know and use daily characterizes it more honestly than measuring it against a

system they meet for the first time. These platform measures remain characterizations, however; the validation outcome proper is the analysis's behavior on the commands participants cause (Section 3.5(e)).

Spanning novice to power users lets the validation study show whether the analysis's decisions hold both for people who can read the commands it judges and for people who cannot, and lets it characterize the platform across both. The validation data accordingly serve two purposes. The system performance data, including the analysis's decision on every command participants cause, validate the analysis in use: they show whether its verdicts and captures hold on commands the corpus does not contain, and whether undo restores what participants' commands changed. The user study metrics and benchmark coverage characterize the platform on which the analysis is deployed. The research questions themselves are answered against the labeled command corpus described in Section 3.1.

1.4 Participant screening criteria

Screening settles who qualifies for the study and where each participant stands in computing proficiency. Each volunteer completes a screening questionnaire online at sign-up, and the behavioral screener below in a short call on their own computer, before any session is scheduled, so that every scheduled session yields an analyzed participant. The questionnaire assesses the following dimensions:

- Years of general computer use
- Primary OS — the operating system used as the primary daily computing environment (Windows / macOS / Linux); must average four or more hours of use per day to qualify
- Self-reported proficiency on primary OS (basic / intermediate / advanced)
- Self-reported command-line familiarity (none / basic / intermediate / advanced)
- Prior exposure to conversational AI interfaces — frequency of use (never / occasionally / regularly / daily) and which tools (chatbots, voice assistants, code-completion or agentic coding assistants, other LLM-based tools)

Self-reported proficiency is confirmed with a short behavioral screener, because self-reported proficiency alone is not a reliable classifier: a participant's technical background in one domain (e.g., software development) does not guarantee fluency with a given OS's native file management, process monitoring, and system administration tools, which is the specific competency the proficiency score is intended to capture. The screener consists of three timed mini-tasks performed unaided on the participant's primary OS using its native tools, one from each of three of the four system-task categories used in the main study: file organization (locate and relocate a set of recently modified files), process monitoring (identify the application consuming the most memory via the OS's native system monitor), and application management (determine whether a named application is installed and identify its version). The facilitator scores each task pass/fail against a fixed time limit and completion criterion.

Each participant receives a **proficiency score** from 0 to 8: the number of behavioral screener tasks passed (0–3), plus self-reported proficiency on the primary OS (basic 0, intermediate 1, advanced 2), plus self-reported command-line familiarity (none 0 to advanced 3). The behavioral screener and the two self-reports enter at roughly equal weight, and the three components are also analyzed separately as a sensitivity check. Because the score, not a classification, enters the analysis, no participant is excluded for falling between groups or for a self-report the screener does not confirm; the screener corrects the self-report rather than vetoing it. Volunteers are excluded only for eligibility: being under eighteen, or using the primary OS for fewer than four hours a day. For description, participants in the lowest third of the score are referred to as **novice users** and those in the highest third as **power users**. Primary OS is recorded for all participants as a grouping variable in the analysis, and prior conversational AI exposure as a covariate; prior AI exposure is analyzed as an exploratory covariate only (see Section 3.4) rather than a primary grouping variable, since splitting the sample along a third axis would leave subgroups too small to support a hypothesis test.

1.5 Task design

The tasks must be fair to both interfaces: the same everyday goal has to be reachable by conversation and by the participant's usual tools. The custom cross-platform task suite is designed to cover human-level system tasks achievable both through SynapseOS natural language commands and through native tools on the participant's primary OS. Task design follows four principles:

1. **Ecological validity** — tasks reflect actions a real user would perform in their everyday computing environment: finding and organizing files, monitoring system resources, managing processes, and handling text data
2. **Cross-platform equivalence** — all tasks are defined as human-level goals achievable both through SynapseOS natural language commands (mapped to bash) and through native tools on the participant's primary OS (File Explorer, Task Manager, Finder, Activity Monitor, GNOME, or equivalent). The method of achievement differs by platform; the human goal does not
3. **Difficulty stratification** — tasks are rated at three difficulty levels (simple, moderate, complex) based on the number of steps required and the specificity of the target operation
4. **Population appropriateness** — every participant receives the same task set, with task wording written in plain language accessible to novice users and free of OS-specific or CLI-specific terminology

The custom task suite comprises tasks organized into four categories: file search and organization, system and process monitoring, application and package management, and text and data processing. All tasks are defined at the human-goal level — achievable via SynapseOS natural language on the evaluation machine and via native graphical or terminal tools on the comparison machine. Each task is documented with a unique identifier, a natural language prompt as presented to the participant, the expected outcome, and the assigned difficulty level. The full suite — twelve core tasks in two parallel forms, plus four boundary tasks lying outside shell expressibility that are scored

and reported separately — is given in Appendix C. Difficulty calibration and form equivalence are validated during the pilot study and revised as necessary before the main study begins.

Section 2: Procedure / Experiment

The procedure has two parts. The recoverability analysis, the study's contribution, is designed and specified first, in the order of the objectives it serves: its effect-semantics model, its effect-composition algorithm, and its recovery-plan generation. SynapseOS, the experimental platform in which it is deployed, follows, together with the scaffolding that makes the validation study trustworthy: collected data is processed and pseudonymized, the system and instruments are validated, each participant session follows a standardized procedure, and participant welfare is protected throughout.

2.1 System design and implementation

The design comprises the recoverability analysis, specified in (a) to (c), one part per objective, and the experimental platform in which it is deployed, in (d) and (e): a conversational interface layer and an execution pipeline, integrated into a unified session manager that launches over the conventional Linux graphical desktop session rather than replacing it.

a) Effect-Semantics Model (Objective 1)

The first objective replaces a list of dangerous command names with a model of what a command does, and the case for it begins with the limits of the list. SynapseOS began with a command-safety check of the same kind as those of the tools surveyed in Chapter 2, and as the execution pipeline's own classification in (e) below: an enumerated list of named command shapes. A proposed command is matched against patterns known to be destructive — *rm*, *dd*, *mkfs*, truncating redirects — and gated if it matches. That

mechanism is the baseline this study seeks to replace, and its limits are structural, not incidental. It cannot enumerate its domain, because the space of destructive commands is open-ended: extending the list to cover *apt* leaves *snap*, *pip*, *docker rm*, and whatever ships next. It does not compose, because each rule matches a command in isolation, while a generated line routinely chains several operations with differing recovery requirements. And it reasons about names rather than effects: *cp a b* is harmless when *b* does not exist and destructive when it does, a distinction no name-matching rule can draw.

This study therefore proposes an algorithm that derives what a command would do from its syntactic structure and decides recoverability over that derivation. It produces a **verdict** — recoverable, recoverable only with prior capture, or unrecoverable — and, for the middle case, a **recovery plan** naming which pre-images must be captured, and by which mechanism, before the command may run.

The formal object. A command is represented by its *effect set*, not by its name. An *effect* on the filesystem is a triple: a kind, drawn from {*read*, *create*, *write*, *remove*, *metadata*}; a target path, resolved against the working directory in which the command would execute; and the pre-image that would have to be captured to reverse the effect. A change to installed packages or running services is a sixth, pathless kind, a *state* effect, recorded with the inverse command that would reverse it (see *Scope of the model* below). A removal whose very bytes survive at another path, as in a rename, is marked as a *move*: undo moves the data back, so it needs no capture. The effect set of a command is the union of the effects of every leaf command within it, taken over its whole operator structure.

The verdict. The verdict is a predicate over the composed effect set: a command is recoverable when every effect in it has an available mechanism that can restore that effect's pre-image. The verdict is not the contribution, since on silent loss the fail-closed list, which asks before every command it does not recognize as read-only, reaches the same outcome on the commands studied. The contribution lies in the derivation of the effect set, (b), and in the plan computed from it, (c).

Soundness. The analysis is deliberately one-sided: any construct whose effects cannot be resolved statically — a path arriving from command substitution, an unexpanded variable, a glob that may match a block device — is treated as unrecoverable, not assumed benign. The two possible errors are not symmetric and are not traded off against each other: over-approximating destructiveness costs the user an unnecessary confirmation prompt, whereas under-approximating it destroys data that cannot be recovered, which is why the evaluation in Section 3.1 reports the false-negative rate separately instead of folding both directions into one accuracy figure. The soundness claim carries two limits. Resolution runs commands before the user has consented, which is safe only because each is proven to have no effects; and another process can change the filesystem between resolving a target and executing the command, so a capture is of the state at resolution time, not a guarantee about the state at execution time.

Scope of the model. The effect domain is the filesystem, extended to two kinds of state on the target system that have no pre-image to copy but do have an inverse: installed packages and running or enabled services. Rather than re-implement the package manager or the service manager, the analysis asks the system's own tools what the command would do — the package manager's simulation mode, the package database, and the service manager's state queries — and derives from the answer the command that puts the state back: a purge for an install, a versioned install for a removal or upgrade, a stop for a start, a disable for an enable. A change is recoverable only if that inverse exists, so removing a package whose old version no repository still offers is unrecoverable; purging a package also captures its configuration files as removals. Undo runs the inverse, with elevated privilege if the original had it, and prints the command for the user to run if it fails. Three things are not represented: what maintainer scripts and a service's own start-up do beyond the state change, since they are arbitrary programs; network mutations and anything on a remote host; and other package managers, which stay unrecoverable unless plainly read-only. A command whose principal effect is one of these is classified unrecoverable by the soundness rule, which is safe but coarse; this is stated as a limitation of the model rather than deferred to the results.

b) Effect-Composition Algorithm (Objective 2)

The algorithm builds a command line's effect set from its parts. It derives the effects of each single command, carries them up through the operators that join commands into one line, and learns the targets that exist only while the line runs.

Extraction. For each leaf command, effects are derived from the command together with its arguments. The derivation is argument-aware because flags change effects, not merely behavior: *rm -i* differs from *rm -f* in whether removal is certain, *cp -n* cannot overwrite where *cp* can, and *chmod* with and without *-R* differ in how many paths acquire a metadata effect.

Composition. Effects propagate upward through the operators that join commands into one line, by a rule that differs per operator. Sequencing with *;* accumulates effects whether or not the earlier part succeeded. Conjunction and disjunction, *&&* and ||, make later effects conditional on an earlier exit status, so the effect set is a union over the branches that may execute. A pipeline has no filesystem effect of its own, but it determines the exit status that gates whatever follows it. Redirection introduces write effects that appear in no command in the pipeline. Command substitution introduces effects whose targets are computable only at runtime, and *xargs* and *find -exec* multiply a command across an argument set that is not statically known. This propagation, not the per-command derivation, is what an enumerated pattern list has no means of expressing.

The parts of a line are analyzed in order against an overlay of what earlier parts created or removed, so that in *mkdir out && cp a.txt out/* the copy lands in a directory that does not yet exist, and a path the line creates and then deletes needs no capture, because there was nothing there to lose. Only across *&&* does a later part see what an earlier one did as done. After *;*, ||, or a pipe, a later part may run although the earlier one failed, so an earlier removal is treated as possibly not having happened, a later write to that path is captured, and an earlier *cd* leaves the working directory unknown. As in the shell, an assignment or *cd* inside a branch or loop outlives it, and a value on which the

paths that may have run disagree is unknown afterward. Loops are analyzed once per resolved item with the loop variable bound, and *sh -c* strings and transparent wrappers such as *sudo* are analyzed as the command they run.

Composition therefore has two parts: propagation through the operators that join commands (*;*, *&&*, ||, pipes, and redirects), which needs no lookup beyond the rule table, and analysis through the commands that run other commands (wrappers, *find -exec*, *xargs*, *eval*, *sh -c* strings, and loops). The composition ablation of Section 3.2 disables the second and keeps the first; Algorithm 1 marks the lines each ablation switches off.

Resolution. Some targets exist only while the command runs: *rm $(ls *.log)*, *find -delete*, *xargs rm*, *git clean -f*, *tar -x*. Rather than declaring these unknowable, the analysis learns them by running a form of the command that cannot change anything — *find* with each action replaced by a print of what it would have touched, the read-only command that feeds *xargs*, *git clean -n*, *tar -t*, *rsync -n*, the body of a command substitution — and analyzes the command again with the concrete targets in place. Resolution is what makes a capture possible for commands whose targets a static reading cannot name. A resolver runs only if the same analysis proves it has no effects, and then inside the bubblewrap sandbox with the whole filesystem mounted read-only; without bubblewrap, resolution is switched off and the command asks. If a resolver cannot be proven read-only, cannot run, or times out, the targets stay unresolved and the command asks. A resolver is also refused when an earlier part of the same line creates, moves into, writes, or removes anything under what it would read, because listing it beforehand would not show what the command will meet.

c) Minimum-Cost Recovery-Plan Generation (Objective 3)

The recovery plan is a minimum-cost cover of the same effect set: for each effect, the cheapest mechanism sufficient to restore it, with redundant captures eliminated. The selection is non-trivial because cost is not uniform across mechanisms: a hardlink capture is independent of file size where a content copy is proportional to it, a single git object identifier covers any number of tracked-file effects at once, and two

effects on overlapping paths need be captured only once. The mechanisms themselves — content backup, hardlink-based trash, git HEAD capture, permission-metadata capture — are those of the execution pipeline described in (e); the algorithm selects among them and introduces none. Only one removal of pre-existing data needs no capture, a rename or move, because the same data survives at the destination and undo moves it back. Compression and archiving are not moves, because the bytes at the destination are a transformation of the original and moving them back returns the wrong content, so they are captured like any other deletion. Where a list can only refuse, the plan adds a capture that restores.

Why the plan is minimal. Cost is counted as the work and storage a capture needs before the command runs: a hardlink, a permission record, a commit identifier, and an inverse command each cost a constant, and a content copy costs the size of the file. The capture a path needs is fixed by its effects: a removal alone is restored from a hardlink; an overwrite needs a copy of the bytes, because a hardlink shares the very data the write changes; and a change of mode or owner needs a record. The choices for different paths are independent, except for two shared captures, both of which only lower the total: a directory captured whole already covers every removal beneath it, and one commit identifier restores the committed state of every tracked file a repository reset touches, so it is chosen once per repository and only files with uncommitted changes need copies of their own. The total is therefore a sum of per-path costs plus the shared captures. Choosing the cheapest sufficient mechanism for each path minimizes each per-path cost, and dropping the removals a captured directory already covers can only lower the total, so together they give the least total cost. The claim is minimality relative to these mechanisms and this cost model, not over every conceivable way of restoring a system; on a copy-on-write filesystem, for instance, a whole-directory snapshot could cost less than the plan’s copies.

Parts (a) to (c) are stated formally together below, so that the analysis can be checked directly rather than reconstructed from prose.

```
Algorithm 1  Effect analysis and capture planning for a generated command

Input:   command c, working directory wd, capture mechanisms M
Output:  verdict in {RECOVERABLE, RECOVERABLE_WITH_CAPTURE, UNRECOVERABLE}
         plan, a set of (mechanism, target) pairs
State:   scope S: the working directory (initially wd) and variables;
         overlay V of paths created or removed by earlier parts of c;
         V ⊕ E is V updated with the creations, removals, and moves in E;
         doubt(V) makes V's removals, and its creations over removed
         paths, possible only, so a later write there is still captured;
         JOIN(S₁, …) makes unknown what the scopes disagree on
Returns: EFFECTS gives (E, S', V'): the effects, then the scope and
         overlay after node; a line giving only E means (E, S, V ⊕ E)
Marks:   [C] is off in the composition ablation, which returns {UNKNOWN};
         [R] is off in the resolution ablation, where RESOLVE returns ⊥

function EFFECTS(node, S, V)
  if node = (A && B) then                          // B runs only if A did
    (E_A, S_A, V_A) ← EFFECTS(A, S, V)
    (E_B, S_B, V_B) ← EFFECTS(B, S_A, V_A)
    return (E_A ∪ E_B, JOIN(S_A, S_B), V_B)
  else if node = (A ; B), (A || B), or (A | B) then
    (E_A, S_A, V_A) ← EFFECTS(A, S, V)    // B may run although A failed
    S' ← S_A (for |, S), its directory unknown if A ran cd
    (E_B, S_B, V_B) ← EFFECTS(B, S', doubt(V_A))
    return (E_A ∪ E_B, S_B (for ||, JOIN(S_A, S_B); for |, S), doubt(V_B))
  else if node = if, case, or while then
    analyze the condition from (S, V), then each branch or body from
    the state the condition leaves, a while body once under doubt
    return (the union of their effects E, JOIN of their scopes and
            S, doubt(V ⊕ E))                   // a branch may not have run
  else if node has a redirect target t then
    (E, S', V') ← EFFECTS(strip_redirect(node), S, V)
    return (E ∪ {write-or-create(t)}, S', V' ⊕ {write-or-create(t)})
  else if node = (for x in W; do body; done) then          // [C]
    E ← ∅
    for each w in EXPAND(W), in order do   // passes share the state
      (E_w, S, V) ← EFFECTS(body, S[x := w], V)
      E ← E ∪ E_w; V ← doubt(V)     // a pass may fail part-way
    return (E, S, V)
  else if node runs another command: sh -c s, eval, a wrapper
       such as sudo, find, or xargs then                    // [C]
    if its targets are known only at run time then
      T ← RESOLVE(node, S, V)                                 // [R]
      if T = ⊥ then return {UNRESOLVED}
    return EFFECTS(inner command, bound to T if resolved, S, V)
  else if node's targets are known only at run time then
    // $( ), tar -x, unzip, rsync, git clean
    T ← RESOLVE(node, S, V)                                   // [R]
    if T = ⊥ then return {UNRESOLVED}
    return EFFECTS(node with its run-time targets bound to T, S, V)
  else
    // a simple command; argument-aware; no rule gives {UNKNOWN}
    if S's directory is unknown then return {UNRESOLVED}
    return RULE(name, args, S, V)       // cd, assignments update S
  end if
```

```
end function

function RESOLVE(node, S, V)
  // learns run-time targets, changing nothing
  if no read-only sandbox is available then return ⊥
  d ← DRYRUN(node)
  // find with actions replaced by prints; xargs' producer;
  // git clean -n, tar -t, rsync -n; command substitutions
  if an earlier part of c changed anything d reads then return ⊥
  if EFFECTS(d, S, V) has an effect other than a read
     or d cannot run, or does not finish in time then return ⊥
  run d in the read-only sandbox       // only a proven read-only d runs
  return the targets d prints
end function

function VERDICT(E, M)
  if UNRESOLVED ∈ E or UNKNOWN ∈ E
     or some effect has no mechanism in M that restores it
    then return UNRECOVERABLE
  if every effect in E is a read, a create, a move, or falls on a path
     created earlier in c then return RECOVERABLE
  return RECOVERABLE_WITH_CAPTURE
end function

function PLAN(E, M)
  P ← ∅
  if E resets a git repository then
    P ← P ∪ {(commit identifier, repository)}
  for each path p with a write, a metadata change, or a removal
      that is not a move, in path order do
    if p was created earlier in c then continue   // nothing to lose
    if p is tracked and unmodified in a repository whose commit
       identifier is in P then continue     // the commit restores it
    if p's only effect is a removal
       and p lies under a directory already hardlinked in P
      then continue
    m* ← the cheapest mechanism in M that restores p's effects
    // hardlink for a removal, copy for an overwrite, record for a mode
    P ← P ∪ {(m*, p)}
  end for
  for each package or service change in E do
    P ← P ∪ {(inverse, change)}
  end for
  return P
end function
```

Algorithm 1. EFFECTS derives and composes the effect set: it applies rules to the leaves, analyzes wrappers, *sh -c* strings, and loops as the commands they run, and calls RESOLVE for targets known only at run time; RESOLVE learns run-time targets by running only a form of the command proven to have no effects; VERDICT and PLAN decide and cover. UNRESOLVED and UNKNOWN are the sentinels that give the analysis its one-sided soundness: they can only push the verdict toward

UNRECOVERABLE. Apart from a path created earlier in the same line, which held nothing to lose, a move (a removal whose data survives at another path) is the only removal PLAN does not capture.

d) Experimental Platform: Conversational Interface Layer

The conversational surface is designed as a persistent, full-screen chat interface that serves as the primary user-facing interface when SynapseOS is active, with the XFCE session beneath it remaining available as a logged fallback. The CLI and TUI interface modes implementing this surface are complete; the full-screen GUI mode is in development, and the readiness checkpoint governing its use in the study is described in the System Environment Specifications. The interface accepts natural language input, displays system responses and command output, and maintains conversational context across the session. An intent parsing pipeline translates each user utterance into a shell command for execution.

Conversational context is session-scoped: history is held in memory for the duration of the login session and is cleared on logout or reboot, with no persistence layer in the thesis prototype. When accumulated history exceeds a fixed token budget — set well below the 8,192-token context window configured for the intent parsing model, which itself supports up to 32,768 tokens — a rolling window discards the oldest turns to keep the conversation within budget. Verbose command output — such as a full directory listing or search result — is truncated to a compact summary before being appended to history, preventing a single command's output from consuming a disproportionate share of the available context.

SynapseOS is designed around a locally-hosted small language model (SLM) as the default, model-agnostic inference backend, with cloud inference available only as an explicit opt-in. Because the system operates at the OS level — processing every natural language command and observing terminal output, file paths, command history, and interaction patterns — routing inference through a cloud API would give an external provider continuous visibility into the user's computing activity. On-device inference

eliminates this exposure by default. The architecture reserves the option for a user to supply an API key for a supported cloud provider and substitute a hosted frontier model, entirely at their discretion, but this path is not implemented in the thesis prototype. The default configuration requires no API key, no internet connection, and no per-query cost, making SynapseOS fully operational in offline and air-gapped environments.

The primary SLM is Qwen2.5-Coder-3B-Instruct, an open-weights[5] code-specialized model that demonstrated the strongest natural language-to-bash translation accuracy among models below 7B parameters in controlled evaluation [47]. Among the models directly compared in that evaluation, its code-specialized pretraining produced a clear margin over general-purpose models of comparable or larger size — 58% NL2Bash accuracy with prompt engineering, versus 49% for Llama 3.2-3B and 37% for a LoRA-fine-tuned Llama 3.2-1B — attributable to Qwen2.5-Coder's training corpus of 5.5 trillion code tokens against Llama 3.2's general-purpose pretraining. At 4-bit quantization[6], the model requires approximately 1.8 GB of memory and runs on the target Debian 13 (Trixie) hardware without a dedicated GPU, which makes it the model for SynapseOS's minimum tiers: 4 GB for the terminal modes and 8 GB for GUI mode. On machines with more memory a larger model may serve, sized so that the desktop, a web browser, and everyday applications keep the memory they need. The model may be fine-tuned via Low-Rank Adaptation (LoRA)[7] on the NL2Bash corpus[8] and a custom SynapseOS task suite to improve intent parsing. If it is, every NL2Bash item in the evaluation corpus, and every description used for the model-generated partition, is excluded from the fine-tuning data, and the model-generated partition is regenerated with the fine-tuned model, so that the analysis is never scored on commands the model was trained to reproduce.

Accuracy is measured using an execution-based scoring methodology rather than syntax-level string matching, following Westenfelder et al. [47] — see Section 2.3 for the full evaluation procedure.

This selection was reconfirmed against models released after the initial literature review, to guard against the fast-moving small-model landscape becoming stale before study execution; no smaller alternative offered comparable shell-generation evidence within the same CPU-only hardware budget [19]. Which larger model serves each larger tier is decided before the pilot study by a comparison on the development machine described in the System Environment Specifications, scoring candidates on the same natural-language requests by execution and timing them under everyday desktop load. Candidates include larger members of the same family and mixture-of-experts models, whose memory footprint is set by their total parameters but which are expected to run on a CPU closer to the speed of their active parameters, a claim the comparison measures rather than assumes; such models fit only the larger tiers. The study machine's tier, and with it the model participants use, is fixed from that comparison. If that model is not the minimum tier's, the model-generated partition of the corpus (Section 1.2(a)) is regenerated with it, so that the partition remains a sample of what participants' model writes.

e) Execution Pipeline

The execution pipeline is where the analysis takes effect on the user's system. Once the intent parser produces a shell command, SynapseOS passes it through a confirmation gate and an undo journal before execution, following the reversibility-and-guardrail framing proposed for LLM-powered natural language shells [13]. The gate asks only where it must, since a command that takes longer than a caregiver would is among the costs VoicePilot's guidelines for LLM speech interfaces warn against [33]. It acts on the analysis's verdict under one of two policies. Under the *strict* policy, the default and the policy used in the validation study, it asks before every command whose verdict is not plainly recoverable, that is, before every command that is recoverable only with capture or unrecoverable, and on approval takes the captures the plan names before the command runs. Under the *capture* policy it asks only before unrecoverable commands, and takes the plan's captures for the rest without asking. Under either policy

the enumerated-pattern list is still consulted, and a command it flags also asks, so the list can add a confirmation but never remove one; with the analysis disabled, the gate falls back to the list alone. Captures use five mechanisms — full content backup, hardlink-based trash, git HEAD capture, permission-metadata capture, or, for package and service state, an inverse command — and are recorded to a persistent undo journal before the command runs, so that a command approved in error remains recoverable; rollback is invoked through a dedicated undo command rather than from within the conversation. Once the gate has passed the command, it is dispatched to a bash subprocess, and its standard output and standard error are captured and rendered in the conversational surface as the system's response to the original natural language input.

SynapseOS executes only shell-expressible operations. The system's capability boundary is the Linux command-line toolchain: file management, process control, text processing, package management, network configuration, and any application that exposes a command-line interface. GUI-only interactions — clicking buttons in graphical applications, navigating web page elements, or interacting with visual-only interfaces — are explicitly outside scope, in deliberate contrast to broader desktop agent frameworks that additionally orchestrate GUI automation atop the existing session [54]. This boundary is intentional: it constrains the research to a tractable, verifiable claim that the system can be built, evaluated, and defended within the thesis timeline.

2.2 Data pre-processing

Collected data is put into a consistent, analyzable form, with participants' identities replaced by codes, before any statistics are computed. Prior to statistical analysis, the command corpus and all data collected during the benchmark evaluation and user study — not the system itself — undergo the following processing steps:

- **Command corpus:** each candidate command is given a fixture built from its own text before the analysis is consulted; commands that do not finish cleanly on their fixture are excluded and counted with the reason, never labeled by guess

(Section 1.2(b)); and the corpus is fixed by a recorded seed and hash before the analysis is run on it (Section 3.1(e))

- **Task normalization:** task descriptions and expected outcomes are standardized across both the OSWorld benchmark and the custom task suite to ensure consistent evaluation criteria
- **Log parsing and event extraction:** raw session logs from SynapseOS telemetry are parsed to extract per-task metrics — completion time, error events, and undo events
- **Metric computation:** task completion time is computed as the elapsed time between task prompt presentation and task completion event. Error rate is computed as the ratio of failed or incorrect outcomes to the number of tasks presented; the denominator is the task count rather than the attempt count, so that retrying cannot lower a participant's error rate. Each task is scored against a written rubric specifying the observable end state that counts as success, with partial completion, abandonment at the time limit, and facilitator intervention coded as distinct outcomes. Because SynapseOS outcomes are captured by the session logger while native-OS outcomes must be judged from screen recording, all tasks in both conditions are scored by one procedure: the rubric, fixed before any session is run, is applied to the screen recording by a single research team member, and for the SynapseOS condition a script additionally checks the end state the rubric names (a file, a package, a service) from the session log, so that the scoring of that condition does not rest on the scorer's judgment alone. SUS and NASA-TLX scores are computed according to their respective standard scoring formulas
- **Participant data pseudonymization:** all participant records are pseudonymized at the point of extraction, replacing identifying information with participant ID codes prior to any analysis

2.3 Validation

Before the results can be trusted, the system and the instruments that measure it are themselves checked. The following validation procedures are applied to ensure the reliability and accuracy of the system and the study instruments:

- **Pilot study:** conducted with five participants prior to the main study to validate the task set difficulty, the questionnaire instruments, and the session procedure; findings from the pilot study are used to revise instruments and procedures before the main data collection begins
- **Intent parsing accuracy:** evaluated using an execution-based scoring methodology [47] rather than syntax-level string matching: each generated command is executed in a controlled environment, and its resulting system state is compared against a human-annotated expected outcome for each task utterance. Ambiguous cases, where the resulting state differs from the expected one only in ways the task did not ask about, are decided by a written equivalence rubric fixed before data collection and applied independently by two members of the team; disagreements are settled by discussion, and the number of cases decided this way is reported. Accuracy is computed as the proportion of first-attempt matches
- **Confirmation policy effectiveness:** evaluated by verifying that every command in the task set that the analysis judges unrecoverable, and under the strict policy every command it judges recoverable only with capture, triggers the confirmation prompt, and that no such command executes without user approval
- **Undo success rate:** evaluated by executing a representative set of reversible operations and verifying that the undo log correctly restores the prior system state in each case

The last two procedures have been carried out once on the prototype, as a manual verification of ten test cases recorded as terminal logs and provided as supplementary material. In seven of them, a request is sent to the deployed system, where the language model writes the command, and the run is repeated with the analysis

disabled, each from a freshly created test directory of ordinary files. Both configurations include the same undo log, so the analysis changes only what is captured before a command runs. The affected files are listed in the initial state, after execution, and after undo, and in every case the model wrote the same command in both runs. Requests name the kind of command under test, and some spell it out, because with loose phrasing the model produced a usable command of the intended shape in only two of seven cases, a limitation of translation rather than of the analysis. Of the remaining cases, one examines the analysis alone, one uses an everyday request, and one examines the scope of undo. All ten behaved as expected. Without the analysis, two data-destroying commands ran with no confirmation at all; where a command's targets were indirect, undo restored the affected files in every case with the analysis enabled and in none with it disabled; and where targets were named directly, the two configurations behaved identically. The verification is illustrative rather than evidential: its cases were chosen to exhibit specific behaviors, so it supports no claim about rates, and the claims of RQ2 through RQ5 rest on the corpus evaluation in Section 3.1. What it adds is confirmation that the analysis's decisions carry through to the files themselves in the deployed system, which the corpus evaluation, presenting commands directly to the analysis, does not exercise. It also surfaced two defects, recorded in `docs/open-problems.md`; the one that bears on the study is described under threats to validity (Section 3.7).

2.4 Study session procedure

Every session runs through the same steps, from briefing and consent to the closing debrief. Each participant session follows a standardized procedure conducted in the following order:

1. **Briefing:** The session facilitator explains the study purpose, session structure, and participant rights. The participant reads and signs the informed consent form. Screening was completed at sign-up (Section 1.4), so no session time is spent on it.

2. **System familiarization:** For the SynapseOS condition, the participant is given a five-minute orientation to the conversational interface and shown how to enter natural language commands. For the native OS condition, no familiarization is provided — participants use their own operating system as they normally would. No task-specific training is provided under either condition.
3. **Task completion:** The participant completes the full task set under the assigned condition. Tasks are presented one at a time via a printed task prompt sheet. The session logger captures start and end times automatically, and screen recording captures all on-screen activity. Participants are not asked to verbalize while tasks are timed, because concurrent think-aloud reliably slows task performance, and here it would slow the two conditions unequally: narrating while composing a natural-language instruction competes for the same linguistic resources far more than narrating while pointing and clicking. Verbal data is instead collected retrospectively, by replaying the screen recording with the participant during the post-study interview.
4. **Post-condition questionnaire:** Immediately after completing all tasks under a condition, the participant completes the SUS and NASA-TLX questionnaires for that condition.
5. **Condition changeover:** If the participant has not yet completed both conditions, a short break is offered, the participant moves to the other machine (SynapseOS or native OS), and the session returns to step 2.
6. **Post-study interview:** After both conditions are completed, the facilitator conducts a semi-structured interview of approximately 10–15 minutes covering perceived ease of use, trust in the conversational interface, preference between conditions, and open-ended feedback.
7. **Debrief:** The facilitator discloses the full study objectives, answers participant questions, and thanks the participant. Session data is pseudonymized under the participant's ID code at session close; the ID-to-consent mapping is retained for

the fourteen-day withdrawal window described in Section 2.5 and destroyed thereafter.

Total estimated session duration per participant: 90–120 minutes.

2.5 Ethical considerations

This study involves human participants and is subject to ethics review and approval by the Institutional Review Board (IRB) or equivalent ethics committee of Mapúa University – Makati prior to the commencement of participant recruitment. The following ethical principles are observed throughout the study:

- **Informed consent:** All participants receive a written information sheet describing the study purpose, procedures, data collected, and their rights. Participation is entirely voluntary; participants may withdraw at any time without penalty or consequence.
- **Pseudonymization:** Each volunteer is assigned an ID code at screening, which is kept on enrollment, and the screening answers, all session logs, questionnaire responses, and transcripts are stored and analyzed under that code alone. Recordings are pseudonymous rather than anonymous: voice is inherently identifying and screen video may capture file names visible on screen, so they cannot be anonymized at the point of collection and are instead governed by the retention rule below.
- **Data minimization:** Only the data types specified in Section 1.1 are collected. No identifying metadata — names, email addresses, or device identifiers — is retained beyond the consent confirmation step.
- **Data security:** All collected data is stored on password-protected institutional infrastructure accessible only to the research team. Interview recordings are deleted after transcription and verification.
- **Confidentiality:** Study findings are reported in aggregate. No individual participant’s data is disclosed in any form that could enable identification.

- **Risks and protections:** Participants issue natural-language instructions that cause real commands to execute, including commands capable of deleting or modifying files. That risk is contained by the study environment rather than by restricting what participants may attempt. Both conditions run on dedicated laboratory machines provisioned for the study, holding only synthetic task data and no personal or institutional files; no participant uses a personal device or signs into a personal account at any point; and each machine is restored to a clean baseline image between participants. Participants are informed before consenting that the commands they trigger execute for real on the study machine and that nothing belonging to them is exposed. The foreseeable discomfort is frustration or a sense of failure while working under observation and timing; participants are told in advance that the interface rather than their competence is under evaluation, and that they may pause or stop without giving a reason.
- **Legal basis and retention:** Personal data in this study is processed under Republic Act No. 10173, the Data Privacy Act of 2012, on the basis of the participant's written consent. Screen recordings and audio are retained only as long as coding requires: both are destroyed once transcription and coding are verified complete, and in no case later than ninety days after the final session. The mapping between ID codes and signed consent forms is stored separately from study data, accessible only to the principal researcher, and destroyed when the withdrawal window below closes, fourteen days after the last session.
- **Right to withdraw:** Participants may discontinue the session at any point, and may request withdrawal of their data for up to fourteen days after their session — the ID-to-consent mapping is retained through that window specifically so that withdrawal remains possible. After it closes the mapping is destroyed and individual records can no longer be located. This limit is stated on the consent form so that it is understood before participation rather than discovered after.

Section 3: System Testing

System testing first evaluates the analysis and then validates it in use. Section 3.1 evaluates it against baselines on the command corpus (Objective 4), answering RQ2 through RQ4; Section 3.2 then measures, by ablation, what composition and runtime-target resolution contribute (Objective 5), answering RQ5. Sections 3.3 to 3.7 define the validation study on the platform: its evaluation dataset, evaluation profile, criteria and formulas, data analysis plan, and threats to validity.

3.1 Evaluation against baselines (Objective 4)

This section is the study's central measurement: how the analysis compares with simpler checks on the same commands. The analysis is evaluated on the command corpus of Section 1.1(a) and 1.2(a)–(b): subsection (a) sets out the baselines, (b) the metrics, and (c) the statistics; (d) reports the results, and (e) the threats specific to this evaluation. The evaluation answers RQ4 directly and supplies the evidence for RQ2 and RQ3. Whether a command's effects can be derived before it runs shows in silent loss and capture coverage, overall and for each command shape; whether the plan restores shows in capture coverage verified by sandboxed undo.

a) Baselines

Three baselines bound the comparison. The first is the enumerated-pattern classifier already implemented in the prototype, which matches commands against a hand-maintained list of known destructive shapes. It is the honest comparison, not a weakened construction, because it is what the system does today. The second turns the same list around to fail closed: every command in the line, including those inside substitutions, loops, and wrappers, must be a known read-only form, or the line asks. It resolves no targets and plans no capture, but it removes the enumerated list's silent failures on commands nobody named, which makes it the strongest simple alternative. The third, the upper bound on safety, is a full pre-emptive copy of the working directory before every command; it recovers everything and is unusable in practice, and it sets the

cost against which the minimal-plan computation must justify itself. That cost was measured rather than asserted. On this study's ext4 evaluation machine, the cheapest real version of a pre-emptive snapshot — a Timeshift-style hardlink snapshot via `rsync -a --link-dest`, which is what a filesystem with no native snapshot falls back to — was timed against a working directory of 1,165 real files (580 MB). A snapshot took 60–250 ms even when nothing, or exactly one file, had changed since the last one, because deciding what to hardlink means visiting every entry in the tree regardless of what changed. The capture plan for a comparable single-file command took 0–20 ms, since its cost is set by what the command touches rather than by the size of the directory around it (`pilot/snapshot_bench.txt`). A copy-on-write filesystem such as btrfs or XFS would close most of this gap, since it would not need to visit every file either; on ext4, the filesystem this study is conducted on, the gap remains.

Because the first baseline was written by the same team, it was checked against command-safety checks that others ship. Two open-source checks were scored on the same held-out corpus and labels, and their results are reported in (d) beside the others: the dangerous-command check of OpenAI's Codex CLI [32], ported from its source and verified against that source's own unit tests, and cc-safety-net [21], a pre-execution guard used with several agentic coding tools, run unmodified under each of its three presets. On the 325 commands that destroy files, they let between 73% and 97% run with no confirmation and no capture, against 31% for the enumerated-pattern classifier, and neither makes any provision for recovery. Both are designed for coding agents working inside a version-controlled project, where deleting project files is routine and version control is the safeguard; these figures therefore describe how they would serve a person's own files, not how well they serve their intended purpose. What they do establish is that the first baseline is the strictest enumerated list tested, so comparisons against it understate rather than overstate the analysis's advantage over lists in current use. No shipping tool was found that determines a command's effects before it runs in order to capture what it would destroy, nor one that captures, per command and before it runs,

exactly what that command would destroy; that is the gap identified in Chapter 2, Section 2.5. Wholesale snapshot recovery is represented by the pre-emptive-copy upper bound.

b) Metrics

Table 3.3. Metrics for the recoverability algorithm, and what each is for.

Metric	What it measures
Silent loss	Commands labeled recoverable-with-capture or unrecoverable that run with no confirmation and no capture. **Primary.** The failure that destroys a user's data without their being asked
Capture coverage	Of commands labeled recoverable-with-capture, the share for which the system captured before running and the capture restores the prior state, verified by executing the command in a sandbox, undoing it, and comparing the tree. **Primary.** A confirmation that is answered yes and then cannot be undone is counted against this, not credited as safe
False-negative rate	Commands judged recoverable that are not. **Reported on its own rather than folded into an accuracy figure**, because this is the error that destroys a user's data and it is not interchangeable with the error below
Needless confirmation	Of the commands that lose nothing, the share on which the user is asked: under the capture policy, those the analysis judges unrecoverable; under the strict policy, also those it judges recoverable only with capture. The cost is unnecessary friction, which matters because habituation to repeated confirmations makes people stop reading them [4]
Recovery success rate	Of the commands executed in the recovery check, those the analysis did not judge unrecoverable, the proportion whose prior state undo restored exactly — verified by executing the command in a sandboxed copy, applying the undo, and diffing against the pre-state
Storage and time overhead	Cost of the computed plan against the full-snapshot baseline. This is what "minimal" has to earn
Contribution of composition and resolution	The drop in capture coverage and silent loss when composition, runtime-target resolution, or both are switched off with the same rule table — the measure most directly tied to what an enumerated-pattern approach cannot do at all

c) Statistical Comparison Against Baseline

The question here is whether the differences between the analysis and its baselines are larger than chance would produce. The analysis and each baseline are scored against the same labeled corpus, so every comparison is paired rather than independent: for each command, each system either lets it run silently or does not, and either covers a loss or does not, and two systems' outcomes on that one command are not independent of each other. Significance is therefore tested with McNemar's exact test on the resulting 2×2 table of discordant pairs (commands one system handles correctly and the other does not) at $\alpha = 0.05$, rather than with a two-sample test, which would treat the corpus as though two different sets of commands had been drawn.

Silent loss, the safety measure of Table 3.3, is reported on its own rather than folded into an accuracy figure. It is tested over the 405 commands that lose something or act outside the filesystem, the only commands on which it is defined, with a 95% Wilson confidence interval on each system's rate; capture coverage is tested over the 325 executable losses. The Wilson interval is used rather than the normal approximation because silent loss is expected to be low for the stronger systems, and the normal approximation is unreliable near the boundary of a proportion, which is exactly where this metric sits. For the analysis, the false-negative rate of Table 3.3 coincides with silent loss, because a command it judges recoverable runs without being asked about.

A significance test states only whether two methods differ, not by how much, so each comparison is paired with an effect size, the paired-data counterpart of the Cohen's d reported for the user study (§3.5(d)). The primary measure is the **risk difference**: the algorithm's accuracy minus the baseline's, with a Newcombe-adjusted 95% confidence interval built for paired proportions, reported for silent loss and for capture coverage. The **odds ratio** from the same 2×2 table of discordant pairs is reported alongside it, because it is the effect-size convention most directly tied to McNemar's test and lets the result be compared with other paired-classifier studies that report it in preference to a risk difference.

With three baselines and two primary metrics, the comparisons that carry a claim are the analysis against each of the first two baselines on silent loss and on capture coverage. These four tests form one family, so significance is judged with a Holm correction at a family-wise $\alpha = 0.05$, and the uncorrected values are reported beside the corrected ones. Needless confirmation is tested as a second family of its own: the analysis against the pattern list and against the fail-closed list, under each policy, by McNemar's exact test with a Holm correction across the four. The open-source checks are compared with the analysis on silent loss by McNemar's exact test and reported with Wilson intervals, outside the four-test family, because they are a check on the first baseline rather than a primary comparison. The full-snapshot baseline is a bound on cost and safety, not a comparison of accuracy, and is reported descriptively. The remaining metrics of Table 3.3 are reported where they arise: needless confirmation in (d) and Section 3.2; recovery success, as the share of executed commands the undo restored exactly, in (d); storage overhead from the recovery check, where the median capture over the 304 commands that captured anything was 0% of a full copy of the fixture, because most captures are hardlinks; time overhead against the snapshot baseline in (a); and the contribution of composition and resolution through the ablations in Section 3.2.

d) Results on the Held-Out Corpus

The procedure above has been run on the prototype, so that the proposal rests on a measurement and not only on a design. Four corpora were generated from different seeds and numbered rounds 4 to 7, after the prototype's earlier evaluation rounds, and each was drawn after fixing what the recovery check on the one before it exposed. Round 4 (778 commands, seed 20260922) came first. Round 5 (772 commands, seed 20260923) drove the recovery check and led to fixing two undo defects. The recovery check on round 6 (776 commands, seed 20260924) found five further undo defects, which were then fixed: a hard link left in place after it should have been removed, the backup files *sed* and *mv* make with a suffix option, *cp --parents*, and a resolver for Perl's *rename* that this was the first corpus to exercise. These three are development corpora.

The figures below are from round 7 (777 commands, seed 20260925: 450 from NL2Bash, 247 from templates, 80 external; 325 executable losses), drawn after the analysis and the undo were both frozen, with the analyzer's hash and the corpus's hash recorded, and run once.

On this corpus the enumerated-pattern classifier let 167 of the 405 commands that lose something or act outside the filesystem run with no confirmation and no capture (41.2%, 95% Wilson interval 36.5–46.1%), while neither the fail-closed list nor the effect analysis let any through (0.0%, interval 0.0–0.9%). Capture coverage, the second primary measure, counts a command as covered only if the analysis planned a capture and the undo then restored the tree exactly, checked by executing the command in the sandbox, applying the undo the runtime would apply, and comparing the tree. By that measure the analysis covered 291 of the 325 executable losses (89.5%, interval 85.7–92.4%), the pattern list 135 (41.5%, 36.3–47.0%), and the fail-closed list none. The pattern list's captures were not executed, so its 135 counts every loss it planned a capture for as restored; this is an upper bound, which makes the comparison in Table 3.4 conservative toward the list. The analysis asked about all 34 losses it did not cover: 9 run code in an interpreter or through *awk*'s *system()*, 8 run a program it does not model, 5 depend on a variable with no value, 4 are *shred*, 4 feed *xargs* from a here-string, 2 are a *find -exec gzip* writing inside the tree it searches, 1 removes its own working directory, and 1 is *rsync* over *ssh*. Across all 631 commands that were executed and that the analysis did not refuse, the undo restored 630 exactly (99.8%). The one failure, *rsync -ur dir_a dir_b && rsync -ur dir_b dir_a*, copies in its second call a directory that the first call's own output had just nested inside it, one level deeper than the analysis re-resolves; it is reported, not fixed (`docs/open-problems.md`).

The two also differ in how often they ask. The fail-closed list asked on every one of the 325 executable losses, capturing nothing for any of them, and on 101 of the 372 commands that lose nothing (27%). The analysis asked on 32 of those 372 under the capture policy (9%) and 45 under the strict policy (12%), and the pattern list on 30 (8%). By McNemar's exact test, the analysis asks far less than the fail-closed list under either

policy (Holm-adjusted p = 2.3×10^{-18} under the capture policy and 9.5×10^{-15} under the strict one), but not less than the pattern list (adjusted p = 0.89 under the capture policy and 0.13 under the strict one). RQ4's third part is therefore answered yes against the fail-closed list and no against the pattern list: the analysis is not shown to ask more often than the list, and in exchange it lets nothing run silently that the list lets through. The two policies differ only in whether the user is asked, so capture coverage is the same under both; the ablations in Section 3.2 are read under the capture policy. These figures score the analysis alone. The deployed gate also consults the pattern list and asks when either does (Section 2.1(e)); scored that way, it asks about 58 of the 372 harmless commands under the capture policy (15.6%) and 67 under the strict policy (18.0%), still well below the fail-closed list's 101, and it still lets no destructive command run silently. The analysis-in-use measures of Section 3.5(e) score the deployed gate.

The pooled pattern holds within each partition. Of the 150 losses among the human-written NL2Bash commands, the analysis let none through silently and covered 133 (88.7%); of the 175 among the templates, which were written by the same team as its rule table, it let none through and covered 158 (90.3%). Under the capture policy it asked needlessly on 8% of the harmless NL2Bash commands and 10% of the harmless templates. The pattern list, by contrast, let 40% of the NL2Bash losses, 23% of the template losses, and 84% of the 80 external commands run silently. The picture is the same by command shape. Of the 405 commands that lose something or act outside the filesystem, the pattern list let through silently 4 of the 34 compound lines, 5 of the 29 hidden effects, 53 of the 164 commands with run-time targets, 37 of the 85 plain commands, 1 of the 13 redirections, and 67 of the 80 external commands. The analysis let none through in any shape; the 34 file-destroying commands it asked about without a capture are 6 compound lines, 8 hidden effects, 15 with run-time targets, and 5 plain commands. The same figures for every system are provided as supplementary material.

Table 3.4. Paired comparisons of the effect analysis against each list on the held-out corpus, with Holm-corrected significance.

Comparison (analysis against)	Discordant items (analysis right / analysis wrong)	p, exact	p, Holm	Risk difference (95% Newcombe)	Odds ratio
Silent loss, pattern list	167 / 0	1.1×10^{-50}	3.2×10^{-50}	+0.412 [+0.365, +0.461]	undefined (no discordant losses)
Silent loss, fail-closed list	0 / 0	1	1	0.000 [−0.009, +0.009]	undefined
Capture coverage, pattern list	170 / 14	3.1×10^{-35}	6.3×10^{-35}	+0.480 [+0.414, +0.540]	12.1
Capture coverage, fail-closed list	291 / 0	5×10^{-88}	2×10^{-87}	+0.895 [+0.856, +0.924]	undefined

The pattern is the one the design predicted. The analysis differs from the pattern list in both silent loss and capture, but from the fail-closed list only in capture: on silent loss the two are indistinguishable at zero. So the verdict is not the contribution; what the analysis adds is a capture that works, at the cost in friction reported above (9% to 12% of harmless commands).

On the model-generated partition the results have the same shape but rest on few losses. The deployed 3B model was asked for a command for each of 350 randomly drawn NL2Bash descriptions. Of the commands it returned, 165 ran cleanly and were labeled, 15 were external, and the rest were excluded because they exited non-zero, timed out, needed a program that is not installed, or wrote outside the fixture. Only 19 of the 165 lost anything, since the model's commands are mostly read-only. On the 19 losses and 15 external commands (34 in all), the pattern list let 20 through silently (58.8%, interval 42.2–73.6%), and the fail-closed list and the analysis one each (2.9%, 0.5–14.9%): a listing that this partition's own external-partition rule mislabeled as device access. The analysis planned a capture that the undo then restored exactly for 17 of the 19 losses (89.5%, 68.6–97.1%), against 10 for the pattern list (52.6%, 31.7–72.7%) and none for the fail-closed list; the undo restored all 149 executed commands. With 19 losses the intervals are wide, and the comparison of coverage against the pattern list is

not significant after correction (Holm-adjusted p = 0.13). The other three comparisons agree with the larger corpus: adjusted p = 1.5×10^{-5} for silent loss against the pattern list, 4.6×10^{-5} for coverage against the fail-closed list, and no difference on silent loss against the fail-closed list. The partition was generated during round 6, the third corpus above, and scored then with that round's analyzer and undo; scoring it again with the final analyzer and its undo, after the seven defects found after round 7 (described below) were fixed, changed none of its 720 decisions (180 commands under each of the four systems scored) and none of these figures.

The two open-source checks were scored on the same 325 commands that destroy files and 372 that lose nothing. Codex CLI's check let 314 of the 325 run without asking (96.6%, Wilson interval 94.0–98.1%) and asked about 1 of the 372 harmless commands (0.3%); cc-safety-net let 262 through under its standard preset (80.6%, 76.0–84.5%), 257 under its strict preset (79.1%, 74.3–83.1%), and 236 under its paranoid preset (72.6%, 67.5–77.2%), asking about 7, 12, and 15 harmless commands respectively (1.9%, 3.2%, and 4.0%). Neither makes any capture, so each has zero capture coverage. Against each, the analysis's advantage on silent loss is significant beyond any reasonable correction (McNemar's exact test, $p < 10^{-70}$, since every discordant command favors the analysis). On needless confirmation, however, the answer to RQ4 against these checks is no: the analysis asks about 32 to 45 harmless commands to their 1 to 15, the cost of letting no destructive command through where they let 73% to 97% through.

After the held-out run, rule-by-rule unit tests, a property test, and two rounds of mutation testing found five further defects in the analyzer, and the review of this chapter found a sixth and a seventh. The sixth was a *find* or a directory removal after an earlier change to the same tree on one line; the seventh, a line read as if every part succeeded, so that a part that runs only after an earlier one failed, as on the right of ||, was analyzed as if it had not. No command in the corpus met the conditions that trigger them; all have been fixed, and re-running the corpus with the fixed analyzer changed none of its decisions under any configuration compared. Because those defects lay where parts of one line interact, a shape the corpus barely samples, a differential test was then built for that

shape: compound lines join the fixture's commands with operators, conditionals, and loops, some on a read-only directory so that parts fail, and every line the analysis lets run is executed in the sandbox, undone, and compared. It found an eighth defect, in undo rather than in the analysis: when part of a line failed or did not run, undo could replay an effect that had not happened, and itself lose a file. With undo fixed to restore by each file's identity, every compound line the analysis let run, 23,054 across six draws, was restored exactly (the last draw restored 6,150 of 6,151 before its one failure was fixed), and the corpus results reported above are unchanged.

Four limits apply to these results. First, the remaining commands the analysis asked on without capturing are mostly ones it cannot see into — an interpreter or an unknown program, a variable with no value in the fixture, a privileged command whose targets it declines to resolve, and *shred* — and it asks on these by construction. Second, a large share of NL2Bash candidates are excluded because they name paths or programs the fixture cannot supply, which biases that partition toward commands that run on a small tree. Third, the external partition is assigned by a table of state-changing tools, not observed, so its labels are only as good as the table, as the model-generated partition's one error shows. Fourth, the model-generated partition was generated in round 6, before the analyzer was frozen; re-scoring it changed no decision, but it was not drawn after the freeze.

e) Threats Specific to This Evaluation

A result is only as trustworthy as the corpus and labels behind it, so this subsection names the ways this evaluation could mislead and how far each is addressed. First, corpus bias is addressed by partitioning, as described in Section 1.2(a). Second, oracle validity replaces label subjectivity. The labels are observed, so they cannot be argued with, but they are facts about one command on one small fixture, run for a few seconds, seeing the working tree and not repository internals; a loss that appears only on a differently shaped tree, or after the time limit, is labeled by what happened here. The oracle's own behavior is pinned by unit tests over commands whose outcome is known,

its exclusions are reported, and the bias they introduce is stated and not corrected: a bias toward commands that run on a small fixture, since most NL2Bash candidates that name paths or hosts the fixture cannot supply are excluded. Third, the algorithm and the labels are produced by the same research team, which creates a risk that the labels reflect the algorithm’s assumptions; against this, the generator is independent of the analysis by construction, and the corpus is fixed by a recorded seed and hash before the analysis is run on it. Fourth, and following from the same fact, the analysis rests on a table of what each known command does, written by the same team, and a table written with the corpus in view measures the cases its author anticipated. The corpus is therefore generated from several seeds: earlier ones are used to correct the analysis and the undo, and the primary result is reported on one drawn after both are frozen, with the analyzer’s hash recorded beside the result. That result is a single draw, so variation from one corpus draw to another is not estimated. The development rounds, drawn from different seeds before the design was frozen, showed the same pattern (zero silent loss for the analysis and capture coverage near 90%), but they were not independent of the design and are not offered as replication. Fifth, the corpus does not verify the package and service part of the model: the oracle can write only inside a fixture, so no command in the corpus changes installed packages or running services under observation, and the inverse plans for such changes are checked by the prototype’s unit tests rather than by execution. Last, a command that exits non-zero is excluded from the corpus (Section 1.2(b)), so a command that destroys part of a tree and then fails is under-represented, and with it the paths on which a later part of a line runs after an earlier part failed, where the seventh analyzer defect lay; the analysis’s handling of those paths rests on its unit tests, and the validation study keeps such commands (Section 3.5(e)).

3.2 Ablation studies (Objective 5)

An ablation switches off one part of a system to measure what that part contributes. Here it separates what the algorithm adds from what the table of known commands beneath it already provides. Chapter 1, Section 1.5 states the analysis’s claim

as what it adds on top of a table of what each known command does, not that it has no such table. This study's adviser raised the distinction directly: how much of the result above comes from the algorithm itself, and how much from that table? The algorithm adds two things a table cannot do, and RQ5 asks for the contribution of each. *Composition* analyzes through the commands that hide another command's targets: wrappers (*sudo*, *env*, *nohup*, *timeout*, and similarly transparent commands), *find* *-exec*/*-execdir*/*-ok*/*-okdir*, *xargs*, *eval*, shell *-c* strings, and *for*-loops. *Runtime-target resolution* learns targets that exist only while a command runs, by running a form of it that is first proven read-only: command substitution and the producer feeding *xargs*, the rewritten *find* expression, and the dry runs of *tar*, *unzip*, *rsync*, *rename*, *git*, and the package and service tools. To separate the two, three further configurations of the analyzer are run over the identical held-out corpus, each identical to the one scored above except for what it disables: composition, routed straight to the unresolved verdict an unknown command receives; resolution, with no runner, so that every target it would have learned is unresolved; and both, which leaves the per-command rule table alone. Every other dispatch path is untouched, including the &&/pipe/subshell traversal that requires no lookup beyond the table. If the result came from the table, disabling either should cost little.

Each costs a great deal (Table 3.5). Capture coverage, verified by the same sandboxed recovery check as the primary result, falls from 291 of 325 executable losses (89.5%) to 115 (35.4%) without composition, to 148 (45.5%) without resolution, and to 104 (32.0%) without both. McNemar's exact test on the paired discordant items rejects equality overwhelmingly in each case, and not one discordant item favors an ablated configuration over the full one: there are 176 such items without composition, 143 without resolution, and 187 without both. Silent loss, by contrast, is unchanged at 0.0% in every configuration. The fail-closed default that the analysis falls back to for anything it cannot resolve depends on neither part: a command it can no longer see through, or whose targets it can no longer learn, is treated exactly as an unknown command already is, which asks rather than runs silently. Composition and resolution are therefore not

what keep the analysis safe; the table and the fail-closed default already do that, which is why Table 3.4 found the analysis indistinguishable from the fail-closed list on silent loss. What they do is make the analysis *useful*: they are the difference between asking about nearly everything and actually recovering what was lost, and that is the part a table, however complete, cannot supply on its own.

The two depend on each other. Neither alone recovers much more than the table: composition without resolution reaches 45.5% and resolution without composition 35.4%, against 32.0% for the table, while the two together reach 89.5%. Of the 143 items lost when resolution is disabled, 126 involve *find* (58 with *-exec*, 47 feeding *xargs*, 12 with *-delete*, and 9 in substitutions or archive commands), 6 are other *xargs* pipelines, 5 are *rsync*, 4 are *git*, and 2 are plain command substitutions: composition can see that *find -exec rm* removes each match, but only resolution can say which files match. The two also differ in what they cost the user. Without composition the analysis asks needlessly about 246 of the 372 commands that lose nothing (66%), because every wrapper and *find* becomes an unknown command; without resolution it asks about 70 (19%), because it still sees what kind of command it faces and only the targets are unknown; the full analysis asks about 32 (9%).

Table 3.5. Composition and resolution ablation: the full analysis against the same rule table with composition, runtime-target resolution, or both disabled, on the held-out corpus.

Disabled	**Coverage (of 325)**	**Silent loss (of 405)**	**Needless asks (of 372)**	**Discordant (full / ablated)**	**p, exact**	**Risk diff. [95% CI]**
Nothing (full)	291 = 89.5%	0 = 0.0%	32 = 8.6%	—	—	—
Composition	115 = 35.4%	0 = 0.0%	246 = 66.1%	176 / 0	2.1×10^{-53}	+0.542 [+0.484, +0.593]
Resolution	148 = 45.5%	0 = 0.0%	70 = 18.8%	143 / 0	1.8×10^{-43}	+0.440 [+0.384, +0.492]
Both (table alone)	104 = 32.0%	0 = 0.0%	251 = 67.5%	187 / 0	1.0×10^{-56}	+0.575 [+0.518, +0.626]

These comparisons answer a different question from the four-test family in Table 3.4 — how much of the result each part of the algorithm contributes, not whether the analysis beats a list — and are treated as a family of their own; with a Holm correction across the three, each p-value remains below 10^{-42}. On silent loss there are no discordant pairs in any comparison. The gap concentrates where the design predicts it should. Commands whose real target is hidden behind a wrapper argument or is not known until the command runs are exactly the cases composition and resolution exist for, while commands whose effects are visible without them — a literal path, a redirect, a chain joined by *&&* — cost the ablated configurations little or nothing, because the table already covers them. In short, the capture-coverage result of Section 3.1(d) is attributable to composition and resolution together, and not to the table they are sometimes mistaken for, while the zero silent loss of Section 3.1(d) is owed to the table and the fail-closed default beneath it.

3.3 Validation study: evaluation dataset

Sections 3.3 to 3.7 define the validation study, in which the analysis is deployed in SynapseOS and exercised by participants. The study exercises the platform on two task datasets: the custom cross-platform task suite described in Sections 1.1 and 1.5, and the OSWorld benchmark task suite [50], which serves as a secondary check related to, but not comparable with, published agent scores, since its tasks are adapted, attended, and gated. The user study involves 40 participants spanning the range from novice users to power users, every one of them analyzed (Section 1.4). The baseline condition is each participant’s primary operating system — Windows 11, macOS, or Linux with GNOME desktop — on a dedicated baseline machine.

3.4 Validation study: evaluation profile

Every participant uses both interfaces, so each serves as their own comparison. The study employs a within-subjects design with two conditions:

- **Condition A:** SynapseOS (conversational interface layer on the Debian 13 evaluation machine)
- **Condition B:** Participant's primary operating system (Windows 11, macOS, or Linux with GNOME desktop, as determined by pre-study screening, on the dedicated baseline machine)

All 40 participants complete both conditions, in counterbalanced order: 20 participants complete Condition A first, and 20 participants complete Condition B first. Condition order is crossed with the two task forms in a 2×2 design of four groups of ten (Section 1.2(d)), which distributes order effects across conditions. The condition × order term of the model in Section 3.6 estimates the period (practice) effect; asymmetric transfer, which a two-period design cannot separate from order, is discussed in Section 3.7.

All participants are analyzed together, with the proficiency score as a continuous moderator, to determine whether the effect of the interface condition differs across experience levels. In addition, primary OS (Windows, macOS, Linux) is recorded as a secondary grouping variable to determine whether results vary across participant backgrounds, and prior conversational AI exposure (frequency of use) is recorded as a covariate to determine whether familiarity with LLM-based tools correlates with SynapseOS performance independent of OS proficiency. For the platform measures, the main analysis is the pooled within-subjects comparison across all 40 participants; per-OS subgroup analyses and AI-exposure correlations are treated as exploratory, given the smaller per-group sample sizes they entail.

3.5 Validation study: criteria and formula

The following metrics and formulas are used in the validation study. Items (a) to (d) characterize the platform against each participant's baseline; item (e) validates the analysis in use.

a) Task Completion Time

Mean task completion time is computed per condition per participant group. Inferential testing and effect size reporting follow the procedures described in Section 3.6.

b) Error Rate

Error rate is computed as the proportion of tasks presented that end in failure or an incorrect outcome:

$$\textit{Error Rate} = (\textit{Failed Tasks} + \textit{Incorrect Outcomes}) / \textit{Tasks Presented}$$

Error events are further categorized by type: intent parsing errors (incorrect bash command generated), execution errors (bash subprocess failure), and recovery failures (an undo that does not restore the prior state). Retries are recorded separately as attempts and do not change the rate, whose denominator is the number of tasks presented, not attempted (Section 2.2). The difference in error rate between conditions is tested by the logistic mixed model of Section 3.6.

c) User Satisfaction

Satisfaction is measured with two established questionnaires, one for how usable the interface feels and one for how much effort it demands. SUS scores are computed per the standard scoring procedure [5] to produce a score on a 0–100 scale; a score above 68 is conventionally treated as above-average usability, a benchmark drawn from scores across a large sample of studies [40]. NASA-TLX scores are computed as the unweighted mean of the six subscale ratings [15] to produce a composite workload score per participant per condition. Because a conversational interface is meant to widen access rather than only to save time, workload and task success are reported with the same prominence as completion time, although completion time alone is the confirmatory test (Section 3.6).

Qualitative interview data is analyzed using thematic analysis to identify recurring themes in participant perceptions of both conditions.

d) Statistical Significance and Effect Size

This subsection sets which comparison is the main test, how the size of a difference is judged, and why forty participants suffice for it. Task completion time is the primary platform measure and is tested at $\alpha = 0.05$; error rate, SUS, and NASA-TLX are secondary measures, tested at a familywise $\alpha = 0.05$ with Holm's correction across the three. Effect size is reported as Cohen's *d*, with thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Sample size is justified by an a priori power analysis for a paired-samples comparison of the primary measure, two-tailed, at $\alpha = 0.05$, computed against the noncentral *t* distribution. Detecting a medium effect ($d = 0.5$) at 80% power requires 34 participants; the 40-participant sample has about 87% power at that effect size and 80% power for effects of $d = 0.46$ or larger, leaving margin for attrition. For the secondary measures, Holm's first test is at $\alpha/3$, where the sample has about 74% power at $d = 0.5$ and 80% power for $d = 0.53$ or larger. Because proficiency enters as a continuous score and no participant is set aside, every comparison uses all forty. Even so, the condition × proficiency interaction, which characterizes whether the platform serves users of different proficiency differently, remains the least-powered comparison in the design, since an interaction of a given size requires substantially more participants than a main effect of the same size. It is therefore reported with effect sizes and confidence intervals as the main descriptive finding for the platform, and is not treated as a confirmatory significance test. These platform comparisons set the sample size; the analysis-in-use measures of (e) are descriptive and need no power analysis.

e) Analysis Decisions in Use

The validation study is also a second test of the analysis, on the commands people actually cause rather than those drawn into a corpus. Every command executed during the SynapseOS condition is logged with the analysis's verdict and capture plan and with a snapshot of the task directory taken immediately before it ran (Section 1.1(d)). Each is then replayed on a copy of that snapshot in the sandbox and labeled as the corpus is (Section 1.2(b)), with one difference: a command that exits non-

zero is kept, not excluded, and labeled from its before-and-after difference, since in use a failed command's partial effects are real. The labeled commands are scored with the corpus measures: *silent loss in use*, the proportion of data-losing commands that ran without confirmation and without a capture; *restoration in use*, the proportion of data-losing commands whose capture, applied by undo, restored the fixture exactly; and *needless confirmation in use*, the proportion of commands that lose nothing on which the participant was nonetheless asked. Each undo a participant invokes is also checked against the state before the command it reverts. A command that changes installed packages or running services, which a directory snapshot cannot hold, is scored instead by recording the affected packages' versions and the services' active and enabled states before it runs and checking them after undo. These are the measures by which the platform validates the analysis: a silent loss here would be a failure of the analysis on a command people actually cause, which the corpus could not reveal.

f) Benchmark Task Success

For the adapted OSWorld tasks, the platform's task success rate is the proportion of tasks whose resulting system state matches the benchmark's expected outcome, of the tasks attempted. The adapted subset, the OSWorld tasks expressible as shell commands, is fixed before data collection and its size reported with the result.

3.6 Validation study: data analysis plan

The plan treats the two kinds of evidence differently: the analysis in use is described, and the platform's study measures are tested. The analysis-in-use measures of Section 3.5(e) are reported descriptively, as proportions with Wilson 95% intervals over the commands executed, and set beside the corpus results of Section 3.1; they are not tested for significance, because the number and kind of commands participants cause cannot be fixed in advance. The platform measures, by contrast, are analyzed in three stages. First, descriptive statistics — means, standard deviations, and 95% confidence intervals — are computed for all platform measures (task completion time, error rate,

SUS score, NASA-TLX score) per condition, and for the lowest and highest thirds of the proficiency score. Second, a Shapiro-Wilk test of each distribution's normality selects the inferential test: the paired-samples t-test for normally distributed data, the Wilcoxon signed-rank test otherwise. Third, effect size is computed for all primary comparisons regardless of statistical significance: Cohen's *d* for the paired-samples t-test and the matched-pairs rank-biserial correlation for the Wilcoxon signed-rank test.

Whether the effect of the interface condition depends on technical background is examined through the interaction between condition and the proficiency score of Section 1.4, estimated on all forty participants; treating proficiency as a score rather than splitting it into two groups keeps the power a split would discard, and the lowest and highest thirds of the score are also shown, for description. Because condition order and task form are crossed between subjects (Section 1.2(d)), each platform measure is modeled with condition, proficiency score, order, and task form as fixed effects, together with the condition × proficiency interaction and the condition × order term, which in a two-period crossover is the period (practice) effect. Error, binary per task, is modeled by a logistic mixed model and completion time by a linear mixed model on its logarithm, both with crossed random intercepts for participant and task; the SUS and NASA-TLX scores, taken once per condition, by a linear mixed model with a random intercept for participant. The condition term of the completion-time model is the confirmatory test; those of the other three models are secondary and Holm-corrected across the three. The paired tests above serve as sensitivity checks, and the power analysis, computed for a paired *t*-test, approximates the power of the condition term.

The benchmark task success rate of Section 3.5(f) is reported descriptively, as a proportion with its Wilson interval, and the telemetry measures of Section 1.1(d) that are not otherwise analyzed (command latency and conversational turns per task) as means with confidence intervals per condition.

Qualitative data from the post-study interviews are analyzed by reflexive thematic analysis, following the six-phase procedure of Braun and Clarke [3]: familiarization with the data, initial code generation, theme search, theme review, theme

definition, and write-up. In keeping with the reflexive approach, which treats the analyst's interpretation as part of the method and does not compute inter-rater agreement, one member of the research team develops the codes and themes; the codebook and the excerpts that support each theme are retained so that the analysis can be audited. Themes are reported with representative participant quotations.

3.7 Validation study: threats to validity

A study of this design can mislead in two ways: its results may not reflect the interfaces themselves, or may not carry beyond the people and setting studied. The following threats to validity are identified, each with the mitigation applied in this study.

Internal validity:

- **Learning effect and asymmetric transfer:** Participants may perform better on the second condition through familiarity rather than interface quality. Counterbalancing alone does not resolve this: it redistributes carryover but cancels it only when transfer is symmetric, which cannot be assumed here, since having solved a task once confers a different advantage in each direction. *Mitigation:* parallel task forms matched on category, difficulty, and step count are crossed with condition order in a 2×2 design, so no participant meets the same task twice; task order within a form is randomized; and condition order is entered as a between-subjects factor in the analysis, with the condition × order term estimating the period (practice) effect. Asymmetric transfer itself is aliased with the order effect in a two-period crossover and cannot be estimated separately; it is limited by the parallel forms and is a stated limitation.
- **Demand characteristics:** Participants may behave differently from natural use because they know they are being observed. *Mitigation:* participants are instructed to complete tasks as they naturally would; the specific hypotheses of the study are withheld until the post-study debrief to reduce social desirability bias.

- **Fatigue:** Completing the full task set twice in a single session may degrade performance in the second condition independent of interface quality. *Mitigation:* a short break is offered between conditions and total session length is bounded to approximately 120 minutes.
- **Fallback reliance:** GUI mode provides a participant-accessible fallback to the underlying XFCE session if SynapseOS becomes unresponsive or a participant wishes to stop using it mid-task; unrestricted, this would let a participant's own choice substitute for the interface under evaluation, confounding the within-subjects comparison. *Mitigation:* every fallback invocation is logged with participant ID, task ID, and timestamp; any task during which it is invoked is excluded from the primary completion-time and error-rate comparison for that task rather than counted as a SynapseOS success, and fallback-invocation rate is reported separately as an exploratory reliability metric.
- **Single scorer and analyst:** Task outcomes on the native-OS condition, and the qualitative themes, are judged by one team member, so their reliability cannot be checked against a second judgment. *Mitigation:* the rubric is fixed before data collection and applied identically to both conditions, the SynapseOS condition is additionally checked by script, and the codebook and supporting excerpts are retained for audit; a residual scorer effect nonetheless remains and is not estimated.
- **Too few data-losing commands:** participants may cause few commands that destroy anything, which would leave the analysis-in-use measures of Section 3.5(e) with small denominators. *Mitigation:* the task suite is reviewed before the pilot to include tasks whose natural solutions delete, move, or overwrite files, and every analysis-in-use proportion is reported with its denominator and Wilson interval, however small.
- **Unscoped undo:** the undo command reverts the most recent entry in the undo log, even one recorded in another directory or an earlier session. The manual verification (Section 2.3) showed that after a task whose only destructive step

was declined, undo proposed reverting an unrelated earlier change. Undo previews what it will revert and requires confirmation, but a participant who confirms without reading the preview could revert the wrong change, which would confound the undo measures the validation study reports. *Mitigation:* before data collection, undo will either be restricted to the current directory and session or the log will be reset at the start of each session; every undo invocation, and the entry it reverted, is logged.

External validity:

- **Sample generalizability:** The study uses a university-recruited sample of 40 participants across three primary OS backgrounds (Windows, macOS, Linux), which limits direct generalization to the broader population of computer users. Findings should be interpreted as indicative rather than definitive; replication with larger and more diverse samples is recommended as a direction for future work.
- **Task generalizability:** The evaluation is bounded by OSWorld coverage and the custom task suite developed for this study. Computing workflows outside this coverage are not evaluated, and results may not extend to all possible use cases.
- **Platform specificity:** SynapseOS is evaluated on a single hardware and software configuration — a dedicated machine at one memory tier, with that tier's model, running SynapseOS in GUI mode (XFCE desktop environment, X11 session) on a Debian 13 (Trixie) base. Other tiers and their models are compared on the development machine but not validated with people, and performance characteristics on different hardware, display configurations, or Linux distributions may differ from the reported results.

Notes

1. Command-Line Interface mode: a single non-interactive invocation, with no session persisting between separate calls, that runs a bounded multi-step loop toward one natural-language request — each step proposed, classified for reversibility, executed (immediately if reversible, after

confirmation if not) through the same gate as the other interface modes, and its result fed back so the model can propose the next step or signal completion — up to a fixed step limit, before the invocation exits. Distinct from TUI mode's persistent interactive chat session; intended for scripting, automation, and one-off remote commands.

2. Terminal User Interface: a text-based interactive interface that runs in a command-line terminal. TUI mode requires no graphical display server or desktop environment and can be accessed remotely over SSH.
3. At its minimum tier the SynapseOS prototype uses Qwen2.5-Coder-3B-Instruct, a code-specialized open-weights model selected for its NL2Bash accuracy. See Section 2.1(d) for the selection rationale, the benchmark evidence, and how models for larger tiers are chosen.
4. An inference server loads a language model into memory and processes query requests from the application layer. Ollama is an open-source tool for running language models locally on consumer hardware without cloud connectivity.
5. Open-weights models make their trained parameters publicly available for local download and use; users run inference on their own hardware without routing queries to an external service provider.
6. Quantization reduces the numerical precision of model weights to lower memory requirements. At 4-bit precision, a model that would otherwise require dedicated GPU memory can run on a standard CPU with modest RAM.
7. Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method that introduces a small number of trainable weight matrices while keeping the base model frozen, enabling task-specific adaptation on consumer hardware without full retraining.
8. NL2Bash is a benchmark dataset pairing natural language command descriptions with their corresponding bash shell commands, used to evaluate a model's accuracy in translating natural language into executable commands.

REFERENCES

[1] Alibaba Cloud. 2026. Alibaba Cloud Linux 4 Agentic Edition (ANOLISA): Product overview. Product documentation. https://help.aliyun.com/en/alinux/agentic-os Accessed 12 October 2026.

[2] B. Becker, N. Jeannerod, C. Marché, Y. Régis-Gianas, M. Sighireanu, and R. Treinen. 2022. The CoLiS platform for the analysis of maintainer scripts in Debian software packages. *International Journal on Software Tools for Technology Transfer* 24, 5 (2022), 717–733.

[3] V. Braun and V. Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.

[4] C. Bravo-Lillo, L. Cranor, S. Komanduri, S. Schechter, and M. Sleeper. 2014. Harder to Ignore? Revisiting Pop-Up Fatigue and Approaches to Prevent It. In *Proceedings of the Tenth Symposium on Usable Privacy and Security (SOUPS 2014)*. USENIX Association, 105–111. https://www.usenix.org/conference/soups2014/proceedings/presentation/bravo-lillo

[5] J. Brooke. 1996. SUS: A “quick and dirty” usability scale. In *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, I. L. McClelland, and B. Weerdmeester (Eds.). Taylor & Francis, 189–194.

[6] A. B. Brown and D. A. Patterson. 2003. Undo for operators: Building an undoable e-mail store. In *Proceedings of the 2003 USENIX Annual Technical Conference*. USENIX Association. https://www.usenix.org/conference/2003-usenix-annual-technical-conference/undo-operators-building-undoable-e-mail-store

[7] L. Clark, N. Pantidi, O. Cooney, P. R. Doyle, D. Garaialde, J. Edwards, B. Spillane, E. Gilmartin, C. Murad, C. Munteanu, V. Wade, and B. R. Cowan. 2019. What makes a good conversation? Challenges in designing truly conversational agents. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI ’19)*. ACM, Article 475, 12 pages.

[8] P. Cousot and R. Cousot. 1977. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference Record of the Sixth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’77)*. ACM, 238–252.

[9] J. W. Creswell and V. L. Plano Clark. 2018. *Designing and Conducting Mixed Methods Research* (3rd ed.). SAGE Publications.

[10] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

[11] A. Goel, K. Po, K. Farhadi, Z. Li, and E. de Lara. 2005. The Taser intrusion recovery system. In *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles (SOSP ’05)*. ACM, 163–176.

[12] M. Greenberg and A. J. Blatt. 2020. Executable formal semantics for the POSIX shell. *Proceedings of the ACM on Programming Languages* 4, POPL, Article 43 (2020).

[13] B. R. Gyawali, S. Achalla, K. Kallas, and S. Kumar. 2025. NaSh: Guardrails for an LLM-Powered Natural Language Shell. arXiv:2506.13028

[14] W. Han, Z. Zeng, J. Huang, S. Jiang, et al. 2025. GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. arXiv:2506.11127v1

[15] S. G. Hart and L. E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Human Mental Workload*, P. A. Hancock and N. Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, 139–183.

[16] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, and J. Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of CVPR 2024*.

[17] Z. Hui, Y. Li, D. Zhao, T. Chen, C. Banbury, and K. Koishida. 2025. WinClick: GUI Grounding with Multimodal Large Language Models. arXiv:2503.04730

[18] E. L. Hutchins, J. D. Hollan, and D. A. Norman. 1985. Direct manipulation interfaces. *Human–Computer Interaction* 1, 4 (1985), 311–338.

[19] J. Jacobs, J. Lapon, and V. Naessens. 2026. Local LLMs for NL2Bash: A Large-Scale Open-Source Model Evaluation for Bash Command Generation. In *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026)*.

[20] P. Jouvelot and D. K. Gifford. 1991. Algebraic reconstruction of types and effects. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '91)*. ACM, 303–310.

[21] kenryu42. 2026. *cc-safety-net*. https://github.com/kenryu42/cc-safety-net Version 2.6.1.

[22] E. Lamprou, T. Zhu, D. Jin, G. Ntousakis, G. Liargkovas, C. Eng, K. Kallas, M. Greenberg, and N. Vasilakis. 2026. Controlling opaque-component effects with semisolates and try. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association. https://www.usenix.org/conference/osdi26/presentation/lamprou

[23] Y. Li, J. He, X. Zhou, Y. Zhang, and J. Baldridge. 2020. Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of ACL 2020*. 8198–8210.

[24] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. European Language Resources Association.

[25] R. Liu, T. Zhe, D. Wang, Z. Yao, K. Liu, Y. Fu, H. Liu, and J. Pei. 2026. AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem. arXiv:2603.08938

[26] J. Lu, Z. Zhang, F. Yang, J. Zhang, L. Wang, C. Du, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang. 2024. AXIS: Efficient Human-Agent-Computer Interaction with API-First LLM-Based Agents. arXiv:2409.17140

[27] E. Luger and A. Sellen. 2016. “Like having a really bad PA”: The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI ’16)*. ACM, 5286–5297.

[28] K. Mazurak and S. Zdancewic. 2007. ABASH: Finding bugs in bash scripts. In *Proceedings of the 2007 Workshop on Programming Languages and Analysis for Security (PLAS ’07)*. ACM.

[29] S. Mehendale and A. Walishetti. 2024. DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation. arXiv:2411.12214

[30] R. Niu, J. Li, S. Wang, Y. Fu, X. Hu, X. Leng, H. Kong, Y. Chang, and Q. Wang. 2024. ScreenAgent: A Vision Language Model-driven Computer Control Agent. arXiv:2402.07945

[31] D. A. Norman. 1986. Cognitive engineering. In *User Centered System Design: New Perspectives on Human-Computer Interaction*, D. A. Norman and S. W. Draper (Eds.). Lawrence Erlbaum Associates, 31–61.

[32] OpenAI. 2026. *Codex CLI*. https://github.com/openai/codex Commit 1f4c473.

[33] A. Padmanabha, J. Yuan, J. Gupta, Z. Karachiwalla, C. Majidi, H. Admoni, and Z. Erickson. 2024. VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST ’24)*. ACM.

[34] J. Park, P. Tang, S. Das, S. Appalaraju, K. Y. Singh, R. Manmatha, and S. Ghadar. 2025. R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL*

2025. 9669–9685.

[35] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel. 2009. Operating system transactions. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP '09)*. ACM, 161–176.

[36] C. Rawles, A. Li, D. Rodriguez, O. Riva, and T. Lillicrap. 2023. Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

[37] O. Rodeh, J. Bacik, and C. Mason. 2013. BTRFS: The Linux B-tree filesystem. *ACM Transactions on Storage* 9, 3, Article 9 (2013), 32 pages.

[38] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In *Proceedings of ICLR 2024*.

[39] D. S. Santry, M. J. Feeley, N. C. Hutchinson, A. C. Veitch, R. W. Carton, and J. Ofir. 1999. Deciding when to forget in the Elephant file system. In *Proceedings of the Seventeenth ACM Symposium on Operating Systems Principles (SOSP '99)*. ACM, 110–123.

[40] J. Sauro and J. R. Lewis. 2016. *Quantifying the User Experience: Practical Statistics for User Research* (2nd ed.). Morgan Kaufmann.

[41] B. Shneiderman. 1983. Direct manipulation: A step beyond programming languages. *Computer* 16, 8 (1983), 57–69.

[42] B. Shneiderman. 2000. The limits of speech recognition. *Commun. ACM* 43, 9 (2000), 63–65.

[43] B. Shneiderman and P. Maes. 1997. Direct manipulation vs. interface agents. *interactions* 4, 6 (1997), 42–61.

[44] J.-P. Talpin and P. Jouvelot. 1994. The type and effect discipline. *Information and Computation* 111, 2 (1994), 245–296.

[45] H. G. W. van Dam. 2025. A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. arXiv:2510.06223

[46] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang. 2024. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. arXiv:2401.16158

[47] F. Westenfelder, E. Hemberg, M. Tulla, S. Moskal, U.-M. O'Reilly, and S. Chiricescu. 2025. LLM-Supported Natural Language to Bash Translation. arXiv:2502.06858

[48] J. Whiteside, S. Jones, P. S. Levy, and D. Wixon. 1985. User performance with command, menu, and iconic interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '85)*. ACM, 185–191.

[49] R. Wilensky, D. N. Chin, M. Luria, J. Martin, J. Mayfield, and D. Wu. 1988. The Berkeley Unix Consultant Project. *Computational Linguistics* 14, 4 (1988), 35–84.

[50] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

[51] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR 2023*.

[52] M. Yuan, Z. Zhou, X. Xiong, et al. 2026. OSWorld 2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. arXiv:2606.29537

[53] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang. 2024. Large Language Model-Brained GUI Agents: A Survey. arXiv:2411.18279

[54] C. Zhang, H. Huang, C. Ni, J. Mu, S. Qin, S. He, L. Wang, F. Yang, P. Zhao, C. Du, L. Li, Y. Kang, Z. Jiang, S. Zheng, R. Wang, J. Qian, M. Ma, J. Lou, Q. Lin, S. Rajmohan, and D. Zhang. 2025. UFO^2: The Desktop AgentOS. arXiv:2504.14603

[55] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu. 2023. AppAgent: Multimodal Agents as Smartphone Users. arXiv:2312.13771

[56] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of ICML 2024*.

[57] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of ICLR 2024*.

[58] X. Zhou, S. Zhang, Y. Zhao, J. Wei, T. Song, A. Cohan, and C. Zhao. 2026. GUI vs. CLI: Execution Bottlenecks in Screen-Only and Skill-Mediated Computer-Use Agents. arXiv:2606.24551

APPENDICES

This section contains the study instruments administered to participants and the procedure governing their administration. Wording, difficulty calibration, and task-form equivalence are validated during the pilot study described in Chapter 3, Section 2.3 and revised before the main study begins; the instruments are reproduced here in the form submitted for ethics review.

APPENDIX A

System Usability Scale (SUS)

Administered immediately after each condition, referring to the interface just used. Items are the standard ten-item instrument of Brooke [5], with the word “system” replaced throughout by “interface” so that participants rate the interaction surface rather than the machine. Each item is answered on a five-point scale from 1 (strongly disagree) to 5 (strongly agree).

Participant ID: _____ **Condition:** ☐ SynapseOS ☐ Own operating system **Date:** _____

#	Statement	1	2	3	4	5
1	I think that I would like to use this interface frequently.	☐	☐	☐	☐	☐
2	I found the interface unnecessarily complex.	☐	☐	☐	☐	☐
3	I thought the interface was easy to use.	☐	☐	☐	☐	☐
4	I think that I would need the support of a technical person to be able to use this interface.	☐	☐	☐	☐	☐
5	I found the various functions in this interface were well integrated.	☐	☐	☐	☐	☐
6	I thought there was too much inconsistency in this interface.	☐	☐	☐	☐	☐

#	Statement	1	2	3	4	5
7	I would imagine that most people would learn to use this interface very quickly.	☐	☐	☐	☐	☐
8	I found the interface very cumbersome to use.	☐	☐	☐	☐	☐
9	I felt very confident using the interface.	☐	☐	☐	☐	☐
10	I needed to learn a lot of things before I could get going with this interface.	☐	☐	☐	☐	☐

Scoring (not shown to participants). For odd-numbered items, subtract 1 from the response. For even-numbered items, subtract the response from 5. Sum the ten adjusted scores and multiply by 2.5, giving a value from 0 to 100. This value is not a percentage; interpretation follows Sauro and Lewis [40].

APPENDIX B

NASA Task Load Index (NASA-TLX)

Administered immediately after each condition, referring to the set of tasks just completed. Six subscales are rated on a twenty-one-point scale presented as a line divided into equal intervals and scored 0 to 100 in steps of five. This study uses the unweighted variant (Raw TLX): the pairwise weighting procedure of the original instrument [15] is omitted, and the composite workload score is the arithmetic mean of the six subscale ratings. All six subscales, including Performance (anchored from Perfect to Failure), run from low to high workload, so no subscale is reverse-scored.

Participant ID: _____ Condition: ☐ SynapseOS ☐ Own operating system **Date: _____**

Subscale	Question	Low anchor	Rating (0–100, steps of 5)	High anchor
Mental Demand	How mentally demanding was the task?	Very low	____	Very high
Physical Demand	How physically demanding was the task?	Very low	____	Very high

Subscale	Question	Low anchor	Rating (0–100, steps of 5)	High anchor
Temporal Demand	How hurried or rushed was the pace of the task?	Very low	____	Very high
Performance	How successful were you in accomplishing what you were asked to do?	Perfect	____	Failure
Effort	How hard did you have to work to accomplish your level of performance?	Very low	____	Very high
Frustration	How insecure, discouraged, irritated, stressed and annoyed were you?	Very low	____	Very high

APPENDIX C

Custom Cross-Platform Task Suite

The suite comprises sixteen tasks in two parts. Twelve *core* tasks span the four categories defined in Chapter 3, Section 1.5 at three difficulty levels, and constitute the main platform comparison. Four *boundary* tasks deliberately fall outside what a shell-mediated interface can express; they are administered and scored but reported separately from the main comparison, because their purpose is to characterize the capability boundary honestly rather than to contribute to the between-condition contrast. Omitting them would leave the suite composed only of work the command line does well, which would make any favourable result difficult to defend.

Every task exists in two parallel forms, matched on category, difficulty, and number of required steps, and differing only in target. No participant encounters the same form twice; form and condition order are crossed as described in Chapter 3, Section 1.2(d). All prompts are phrased as human goals in plain language, containing no shell syntax, no command names, and no interface-specific vocabulary, so that they are equally answerable through natural language on SynapseOS and through native tools on the baseline machine.

Tasks T7–T9 require identical provisioning on all study machines — the same designated applications present or absent, and the two test applications (*SampleApp*, *DemoTool*) installed — so that the prompt means the same thing under both conditions and on all three baseline operating systems; this provisioning is part of the facilitator checklist in Appendix G. Task T9 is classified irreversible and is included specifically to exercise the confirmation gate; T11 mutates file content and is included to exercise the undo journal. Difficulty is assigned by the number of distinct operations required and the specificity of the target, and is validated during the pilot study described in Chapter 3, Section 2.3.

Table C.1. Custom cross-platform task suite — core tasks.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
T1	File search & organization	Simple	Find every PDF in the Documents folder and gather them into one folder named *PDFs*.	Find every image in the Downloads folder and gather them into one folder named *Images*.	All matching files present in the target folder; no matching file left behind or lost.
T2	File search & organization	Moderate	Show the ten largest files anywhere under the home folder.	Show the ten oldest files anywhere under the home folder.	The correct ten files identified, largest/oldest first.
T3	File search & organization	Complex	Find files modified in the last seven days and copy them into a folder named *recent-work*, preserving their folder structure.	Find files larger than 10 MB modified this month and copy them into *large-recent*, preserving their folder structure.	Correct file set copied; directory structure preserved; originals untouched.
T4	System & process monitoring	Simple	Identify the program currently	Identify the program currently	Correct process named.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
			using the most memory.	using the most CPU.	
T5	System & process monitoring	Moderate	Report free disk space, and which folder in the home directory occupies the most of it.	Report free memory, and the three programs consuming the most of it.	Both figures correct at time of measurement.
T6	System & process monitoring	Complex	A process is consuming excessive CPU. Identify it and stop it.	A process is consuming excessive memory. Identify it and stop it.	Correct process identified and terminated; no unrelated process stopped.
T7	Application & package management	Simple	Determine whether VLC is installed, and report its version.	Determine whether GIMP is installed, and report its version.	Correct installation status and version reported.
T8	Application & package management	Moderate	Report how many applications are installed on this machine.	Report which application on this machine was installed most recently.	Correct figure reported, verified against the machine's provisioning record.
T9	Application & package management	Complex	A provisioned test application named *SampleApp* is installed. Uninstall it completely, including the	A provisioned test application named *DemoTool* is installed. Uninstall it completely, including the configuration files it created.	Application and its configuration removed; nothing else removed. **Irreversible — exercises the confirmation gate.**

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
			configuration files it created.		
T10	Text & data processing	Simple	Report how many lines *report.csv* contains.	Report how many lines *sales.csv* contains.	Correct line count.
T11	Text & data processing	Moderate	In *notes.txt*, replace every occurrence of “draft” with “final”.	In *log.txt*, replace every occurrence of “WARN” with “WARNING”.	All occurrences replaced; no other content altered. **Content-mutating — exercises undo.**
T12	Text & data processing	Complex	From *sales.csv*, report total revenue per region, highest first.	From *orders.csv*, report order count per customer, highest first.	Correct aggregation and ordering.

Table C.2. Boundary tasks — outside shell expressibility, reported separately.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
C1	Visual editing	Moderate	Crop the supplied photograph to the person on the left.	Rotate the supplied photograph so the horizon is level.	Result judged correct by the facilitator against a reference image.
C2	Visual comparison	Simple	State which of the two supplied images is brighter.	State which of the two supplied images is more blurred.	Correct image identified.

ID	Category	Level	Form A prompt	Form B prompt	Success criterion
C3	Document formatting	Moderate	Make the heading of the supplied document bold and centred.	Change the supplied document's body font to 14 point.	Formatting applied correctly, verified in the saved file.
C4	Window management	Simple	Arrange the four open windows so all are visible simultaneously.	Minimize every open window except the text editor.	End state matches the requested arrangement.

APPENDIX D

Pre-Study Screening Questionnaire

Completed at sign-up, before any session is scheduled, and used to compute each volunteer's proficiency score and to record the covariates described in Chapter 3, Section 1.4. The form records a participant ID code only; the volunteer's name appears solely on the consent form (Appendix F), which is stored separately from all study data. Screening data belonging to volunteers who are not enrolled is destroyed within seven days of screening.

Screening consent (read to the volunteer before Part 1). This short questionnaire and three small tasks on your own computer tell us how familiar you are with it; every answer is useful to the study. Taking part is voluntary and you may stop at any time. Your answers are kept under a code, not your name. We also ask for one way to contact you, used only to arrange the screening call and your session; it is kept apart from your answers and destroyed within seven days if you are not enrolled, or once your session is booked if you are. Your answers are destroyed within seven days if you are not enrolled. Nothing on your screen is recorded during the tasks. ☐ I agree to be screened

Participant ID: _____ Date: _____

Part 1 — Background (self-report)

1. Are you 18 years of age or older? ☐ Yes ☐ No *(a "No" response ends the screening; the study enrols adults only)*
2. Approximately how many years have you used a computer regularly? _____
3. Which operating system do you use most? ☐ Windows ☐ macOS ☐ Linux
4. Roughly how many hours per day do you use that operating system? _____ *(four or more hours is required for enrolment)*
5. How would you describe your proficiency with that operating system? ☐ Basic ☐ Intermediate ☐ Advanced
6. How would you describe your familiarity with a command line or terminal on any platform? ☐ None ☐ Basic ☐ Intermediate ☐ Advanced
7. How often do you use conversational AI tools such as chat assistants or AI coding assistants? ☐ Never ☐ Occasionally ☐ Regularly ☐ Daily
8. If you use such tools, which ones? _____

Part 2 — Behavioural screener

Three short tasks performed unaided on the volunteer's own primary operating system using its native tools, one from each of three of the four system-task categories used in the main study. The screener exists because self-reported proficiency is an unreliable classifier: fluency with an operating system's own tools is the property being measured, not confidence about it. Each task is scored pass or fail by the facilitator against the stated time limit and completion criterion. The volunteer is told that the screener measures familiarity with their own computer, that every result is useful to the study, and that no outcome affects their participation.

#	Task	Time limit	Pass criterion	Result
S1	Locate the files most recently modified in your home or user folder and move them into a new folder.	3 minutes	Correct files identified and relocated unaided.	☐ Pass ☐ Fail

#	Task	Time limit	Pass criterion	Result
S2	Using your operating system's own system monitor, identify the application currently consuming the most memory.	3 minutes	Correct application named unaided.	☐ Pass ☐ Fail
S3	Determine whether a named application is installed on this machine and report its version.	3 minutes	Correct status and version reported unaided.	☐ Pass ☐ Fail

Proficiency score (facilitator use). Screener tasks passed (0–3), plus Question 5 (Basic 0, Intermediate 1, Advanced 2), plus Question 6 (None 0, Basic 1, Intermediate 2, Advanced 3), for a score from 0 to 8, as defined in Chapter 3, Section 1.4. A volunteer who meets the eligibility rules (Questions 1 and 4) is enrolled whatever the score; no volunteer is turned away for falling between groups.

Proficiency score: _____ / 8 **Eligible:** ☐ Yes ☐ No **Assigned task form:** ☐ A first ☐ B first **Assigned condition order:** ☐ SynapseOS first ☐ Own OS first

APPENDIX E

Participant Information Sheet

Study title. Recoverability Analysis of Language-Model-Generated Shell Commands, Deployed in the SynapseOS Conversational Interface

Who is running this study. This research is conducted by undergraduate researchers in the Department of Computer Science, Mapúa University – Makati, in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science. It will be carried out only after review and approval by the University's Institutional Review Board. Contact details for the research team and for the ethics committee are given at the end of this sheet.

Why we are doing this. Most of what a computer can do is reachable only by people who know how to write commands for it. We have built a system that lets a person say what they want in ordinary language — for example, "find the files I changed

last week and put them in one folder” — and carries it out. We want to find out whether that is genuinely easier to use than the desktop you already use, and whether it makes more difference for people who have never used a command line.

What you would do. You would attend one session of about 90 to 120 minutes. You would complete a set of everyday computer tasks twice: once using our system, and once using the operating system you normally use, on a computer we provide. Which one you start with is decided in advance and is not a test of you. Afterwards you would fill in two short questionnaires about each interface and take part in a 10 to 15 minute conversation about your experience, during which we may replay the screen recording and ask what you were thinking at particular moments.

What we record. We record what happens on the screen during the tasks, audio of the closing conversation, your questionnaire answers, and the times and outcomes of the tasks. We do not record your name against any of this. Everything is filed under a code.

What could go wrong, and what protects you. The instructions you give will cause real commands to run, and some commands can delete or change files. You will be working on a laboratory computer prepared for this study, holding only made-up practice files. You will not use your own device and will not sign in to any of your own accounts, so none of your own data is involved at any point. Each machine is reset to a clean state between participants. Some people find working under observation and a timer uncomfortable, or feel frustrated when a task does not go well. We are evaluating the interface, not you, and you may pause or stop at any moment without giving a reason.

Benefits. There is no direct benefit to you. The findings are intended to inform how conversational interfaces to computers are designed, particularly for people without technical training.

How your information is handled. Your data is processed under Republic Act No. 10173, the Data Privacy Act of 2012, on the basis of your written consent. It is stored under a code rather than your name, on password-protected university infrastructure that only the research team can reach. Screen recordings and audio cannot

be made fully anonymous — a voice identifies a person, and a screen can show file names — so we keep them only as long as we need them for analysis. They are destroyed once transcription and coding are complete, and in any case no later than ninety days after the last session. The list linking your code to your signed consent form is kept separately and destroyed when the withdrawal window below closes, fourteen days after your last session.

Withdrawing. You may stop the session at any point, without explanation and without any consequence. You may also ask us to delete your data for up to fourteen days afterwards; we keep the code-to-consent list for exactly that period so that we can find and remove your records if you ask. After fourteen days that list is destroyed and we can no longer identify which records are yours, so withdrawal is not possible beyond that point.

How the results are used. Results are reported in aggregate in a thesis and possibly in academic publications. We may quote short remarks you make in the closing conversation. Quotations are attributed only to a participant code, never to your name, and you can decline to be quoted while still taking part in everything else.

Questions or concerns. For questions about the study, contact the research team at [research team contact address]. For concerns about how the study is being conducted, or about your rights as a participant, contact the Institutional Review Board of Mapúa University – Makati at [ethics committee contact address], independently of the research team.

APPENDIX F

Informed Consent Form

Study title. Recoverability Analysis of Language-Model-Generated Shell Commands, Deployed in the SynapseOS Conversational Interface

Please read the Participant Information Sheet (Appendix E) before completing this form, and tick each box to confirm your agreement. You may ask questions at any time before signing.

	Statement
☐	I have read and understood the Participant Information Sheet, and I have had the opportunity to ask questions.
☐	I am 18 years of age or older.
☐	I understand that my participation is voluntary, and that I may stop at any time without giving a reason and without any consequence.
☐	I understand that the instructions I give will cause real commands to run on a laboratory computer, that this computer holds only practice files prepared for the study, and that none of my own devices, accounts, or files are involved.
☐	I understand that my data is stored under a code, that the list linking that code to my name is kept separately, and that I may request deletion of my data for up to fourteen days after my session, after which it can no longer be identified.
☐	I understand that some information about the study's specific expectations is withheld until the end of the session, and that it will be explained to me fully before I leave.
☐	I agree to take part in this study.

Separate permissions. You may take part in the study without agreeing to these. Please tick only those you accept.

	Statement
☐	I agree to the screen being recorded while I complete the tasks.
☐	I agree to the closing conversation being audio-recorded, and understand the recording is destroyed once it has been transcribed and checked.
☐	I agree that short anonymous quotations from what I say may appear in the thesis or in academic publications, identified only by a participant code.

Participant name (printed): _____ Signature: _____ Date: _____

Researcher obtaining consent (printed): _____

Signature: _____ Date: _____

Filing note: this is the only study document bearing the participant's name. It is stored separately from all study data, and the code-to-name list derived from it is destroyed on the schedule stated in Chapter 3, Section 2.5.

APPENDIX G

Session Facilitator Checklist

Followed identically for every participant so that session conduct does not vary across the sample. Items marked † must be completed before the participant arrives.

Before the session †

1. Restore both machines to the clean baseline image; confirm no data remains from the previous participant.
2. Boot the baseline machine into the operating system matching the participant's screening record (Windows, macOS, or Linux with GNOME).
3. Provision the task environment identically on both machines: practice file set in place, the applications referenced by tasks T7–T9 present or absent as specified, and the test applications *SampleApp* and *DemoTool* installed.
4. On the SynapseOS machine, confirm the inference server is running and the model is loaded, and run one throwaway task to confirm the system responds.
5. Confirm screen recording works on both machines and has free storage for the full session.
6. Lay out the printed task prompt sheet for the assigned form, the two questionnaire sets, the information sheet, and the consent form.
7. Record which interface mode SynapseOS is running in for this session, per the readiness checkpoint in the System Environment Specifications.

During the session

1. Give the information sheet, answer questions, obtain signed consent. File the consent form separately from all study materials.
2. If the condition about to begin is SynapseOS, give the five-minute orientation to it; the participant's own operating system gets none (Chapter 3, Section 2.4). Use the same wording for every participant; do not demonstrate any task from the task set.

3. Start screen recording. Present tasks one at a time in the randomized order for this participant. Do not prompt, hint, or answer questions about how to accomplish a task.
4. Do not ask the participant to think aloud during timed tasks.
5. Record the outcome of each task and note any facilitator intervention, with the reason.
6. Administer SUS and NASA-TLX immediately on completing the condition, before any discussion of it.
7. Offer a short break. Move to the second machine and repeat from step 2 with the other condition and the other task form.
8. Conduct the closing conversation, replaying portions of the screen recording to prompt recall. Ask what the participant was thinking, not whether they liked the system.
9. Debrief: explain the study's expectations in full, answer questions, and thank the participant.

After the session

1. Transfer all recordings and logs to protected storage under the participant code; delete local copies from both machines.
2. Log any fallback invocation, with participant code, task, and timestamp.
3. Note any deviation from this checklist, however small, for reporting with the results.
4. Restore both machines to the clean baseline image.
5. Diarise the participant's fourteen-day withdrawal deadline.